

从 C++ 到 AI 计算：第三册实践卷

本地量化推理引擎、KV Cache、后端优化与验收

CPU Performance Study

本卷是《从 C++ 到 AI 计算：第三册》的配套实践卷，围绕 `linux_cpu_inference` 贯穿项目展开：模型资产、tokenizer、张量布局、int8 kernel、reference decoder、KV cache、trace、7B 账本、服务调度、CPU/GPU 后端边界和工程验收都要落到可构建代码、测试和报告。

前言

第三册原理卷把 AI 推理系统拆成模型资产、typed graph、张量布局、量化、KV cache、后端、服务运行时和验收体系。本实践卷只做一件事：把这些概念压到同一个可运行项目里，让读者知道每个概念对应哪段代码、哪条命令、哪种输入、哪份报告和哪条失败路径。

贯穿项目是 [books/cpu-volume-3/labs/linux_cpu_inference](https://github.com/secondmind-labs/books/cpu-volume-3/labs/linux_cpu_inference)。它不是玩具目录，而是第三册已经持续维护的本地 CPU 量化推理切片：包含教学模型格式、tokenizer、safetensors manifest gate、int8 linear、packed/AVX2 路径、tiny reference decoder、KV cache trace、7B compute ledger、serving SLO probe、真实 small 开源模型 smoke、CLI、benchmark 和 CTest。实践卷不复制这套代码，避免两份实现漂移；所有练习都回到同一份源码。

本卷的学习顺序是从小证据到大系统：先让一个 tiny MLP 算对，再解释张量地址流和 int8 累加；再让 reference decoder、sampler 和 KV cache trace 固定语义；然后接模型导入、shape verifier、memory planner 和 IR lowering；最后用 CPU/GPU 后端边界、工业专项和回归门禁收束。每章都要求读者交付代码运行结果或报告字段，而不是只抄概念定义。

核心思想

第三册实践的目标不是“写一个看起来像大模型的 demo”，而是建立一条能被复查的推理证据链：资产能校验，shape 能验证，kernel 能对照，trace 能定位，性能能降级，服务指标能解释，回归门禁能长期维护。

常见缩写和术语先导

缩写	全称	本卷中的含义
AI	Artificial Intelligence	这里特指本地推理工程，不讨论泛泛应用口号。
LLM	Large Language Model	decoder-only 文本生成模型是本卷主线。
KV Cache	Key/Value Cache	attention 历史 K/V 状态，属于请求生命周期。
GEMM	General Matrix Multiply	矩阵乘，是预填充和训练常见核心算子。
GEMV	General Matrix Vector Multiply	矩阵向量乘，是单 token decode 的核心形态。
SIMD	Single Instruction Multiple Data	一条指令处理多个 lane 的 CPU 执行方式。
AVX2	Advanced Vector Extensions 2	x86-64 上的 256-bit SIMD 指令集之一。
FMA	Fused Multiply-Add	乘加融合指令，常出现在浮点 kernel 中。
SLO	Service Level Objective	服务目标，例如 TTFT、TPOT、p95/p99。
TTFT	Time To First Token	从请求进入到首 token 输出的时间。
TPOT	Time Per Output Token	decode 阶段每个输出 token 的平均或尾部时间。
IR	Intermediate Representation	编译器内部表示，用于 pass、lowering 和计划生成。
RAG	Retrieval-Augmented Generation	检索增强生成，需要把外部文档和 prompt 编排起来。
MoE	Mixture of Experts	多专家模型，会改变路由、负载和内存账本。
LoRA	Low-Rank Adaptation	低秩适配权重，常用于微调和多租户加载。

这些缩写不要死记。每个缩写在正文里都要落到一个代码对象或报告字段：KV cache 对应 trace 里的 slot 和 position，SLO 对应 serving probe 的 CSV，IR 对应 executable plan，SIMD/AVX2 对应 kernel 反汇编和 benchmark。

本卷结构

本卷按第三册原书的四个大块组织。每一章都对应原书的一组主题文件，并把概念落到同一条实践主线：[linux_cpu_inference](#)。

第一部分从模型资产到量化 kernel。你会运行 tiny MLP，检查 tokenizer 和 safetensors manifest，手算张量地址，比较 scalar、packed 和 AVX2 路径，并解释 int8 scale、accumulator 和 requantize 的边界。

第二部分进入 Transformer、KV cache、sampler 和服务运行时。你会跑 reference decoder trace，解释每个 checkpoint 的 shape、stride、dtype 和 layout，构造 KV cache 状态表，分析 admission、queue wait、TTFT、TPOT、取消和拒绝。

第三部分处理模型导入、真实模型加载闭环、memory planner 和 IR lowering。你会把 manifest、shape verifier、tensor role、state edge、workspace、liveness 和 executable plan 写成可检查报告，并显式运行一个 small 级别开源 GGUF 模型 smoke，而不是只停留在 tiny fixture。

第四部分收束到后端优化和验收。你会比较 CPU kernel 证据、GPU 后端边界、工业专项能力和回归门禁。最终交付不是“跑通一次”，而是可复查的工程档案：命令、输入、输出、trace、benchmark、不可验证边界和回归规则。

目录

前言	ii
常见缩写和术语先导	iii
本卷结构	iv
第一部分 推理链路、张量账本与量化	
第 1 章实践：端到端推理链路和项目总入口	2
一、对应原书问题：概念必须落到对象和命令	2
二、从特例推到规律：先抓一个能手算的切片	2
三、实践任务：把观察固定成报告字段	2
四、验收和递进大作业	2
第 2 章实践：张量布局、地址流与第一个 MatVec	3
一、对应原书问题：概念必须落到对象和命令	3
二、从特例推到规律：先抓一个能手算的切片	3
三、实践任务：把观察固定成报告字段	3
四、验收和递进大作业	3
第 3 章实践：GEMM、packing 与 SIMD 证据	4
一、对应原书问题：概念必须落到对象和命令	4
二、从特例推到规律：先抓一个能手算的切片	4
三、实践任务：把观察固定成报告字段	4
四、验收和递进大作业	4
第 4 章实践：int8 量化、累加器与重新量化	5
一、对应原书问题：概念必须落到对象和命令	5
二、从特例推到规律：先抓一个能手算的切片	5
三、实践任务：把观察固定成报告字段	5
四、验收和递进大作业	5
第二部分 Transformer、KV Cache 与服务运行时	
第 5 章实践：Transformer 切片与 KV Cache 状态	7
一、对应原书问题：概念必须落到对象和命令	7
二、从特例推到规律：先抓一个能手算的切片	7
三、实践任务：把观察固定成报告字段	7
四、验收和递进大作业	7
第 6 章实践：Reference Decoder、logits 与采样	8
一、对应原书问题：概念必须落到对象和命令	8
二、从特例推到规律：先抓一个能手算的切片	8
三、实践任务：把观察固定成报告字段	8
四、验收和递进大作业	8
第 7 章实践：服务调度、SLO 与资源预算	9
一、对应原书问题：概念必须落到对象和命令	9
二、从特例推到规律：先抓一个能手算的切片	9
三、实践任务：把观察固定成报告字段	9
四、验收和递进大作业	9
第 8 章实践：RAG、Agent 与状态图边界	10
一、对应原书问题：概念必须落到对象和命令	10
二、从特例推到规律：先抓一个能手算的切片	10
三、实践任务：把观察固定成报告字段	10
四、验收和递进大作业	10
第三部分 模型导入、AI 编译器与内存计划	
第 9 章实践：模型导入、manifest 与 Shape Verifier	12
一、对应原书问题：概念必须落到对象和命令	12
二、从特例推到规律：先抓一个能手算的切片	12
三、实践任务：把观察固定成报告字段	12
四、验收和递进大作业	12

第 10 章实践：真实模型加载闭环与首 Token Trace	13
一、对应原书问题：概念必须落到对象和命令	13
二、从特例推到规律：先抓一个能手算的切片	13
三、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来	13
四、实践任务：把观察固定成报告字段	14
五、验收和递进大作业	14
第 11 章实践：运行时内存计划、liveness 与 Workspace	15
一、对应原书问题：概念必须落到对象和命令	15
二、从特例推到规律：先抓一个能手算的切片	15
三、实践任务：把观察固定成报告字段	15
四、验收和递进大作业	15
第 12 章实践：IR、Pass、Lowering 与 Executable Plan	16
一、对应原书问题：概念必须落到对象和命令	16
二、从特例推到规律：先抓一个能手算的切片	16
三、实践任务：把观察固定成报告字段	16
四、验收和递进大作业	16
第四部分 后端优化、工业专项与验收	
第 13 章实践：CPU 后端、微内核与性能证据	18
一、对应原书问题：概念必须落到对象和命令	18
二、从特例推到规律：先抓一个能手算的切片	18
三、实践任务：把观察固定成报告字段	18
四、验收和递进大作业	18
第 14 章实践：GPU 后端边界、copy 与同步成本	19
一、对应原书问题：概念必须落到对象和命令	19
二、从特例推到规律：先抓一个能手算的切片	19
三、实践任务：把观察固定成报告字段	19
四、验收和递进大作业	19
第 15 章实践：工业 LLM 专项能力对照	20
一、对应原书问题：概念必须落到对象和命令	20
二、从特例推到规律：先抓一个能手算的切片	20
三、实践任务：把观察固定成报告字段	20
四、验收和递进大作业	20
第 16 章实践：工程验收、回归门禁与最终项目	21
一、对应原书问题：概念必须落到对象和命令	21
二、从特例推到规律：先抓一个能手算的切片	21
三、实践任务：把观察固定成报告字段	21
四、验收和递进大作业	21
完整源码清单	22
一、构建入口和项目说明	22
二、公共合同头文件	38
三、核心实现	44
四、命令行、账本、Trace 和 Benchmark	89
五、测试和模型 Fixture	106
术语表	116

第零部分

推理链路、张量账本与量化

第 1 章实践：端到端推理链路和项目总入口

本章对应原书中的第三册“端到端链路：从模型文件到下一个 token”。不要把它当作概念复述，本章要把模型目录、manifest、tokenizer、typed graph、backend plan、request session、KV cache、oracle 和 profiler 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_mlp_i8.txt`，核心命令或测试入口是 `lcqi_cli`。

Listing 1.1 第 1 章实践：端到端推理链路和项目总入口的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / tiny_mlp_i8.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：只跑一个 tiny MLP 推理请求。读者要先手写预期结果，再运行项目观察输出。动作是：把 `readfiles->tokenize->run_inference->logits->predicted_class` 串起来。如果输出里没有 `predicted_class` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型文件存在不等于推理链路可信；只有 CLI 输出、测试断言、输入合同和失败路径同时成立，才算完成入口闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：最终大作业的总目录：每章报告都放入 `reports/volume3-practice/`，第一章提交项目地图、命令记录和一次成功/失败输入对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 2 章实践：张量布局、地址流与第一个 MatVec

本章对应原书中的第三册“张量布局、地址流与第一个矩阵向量内核”。不要把它当作概念复述，本章要把 TensorView、dtype、shape、stride、row-major、地址公式、bias、ReLU、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `linear_i8`，核心命令或测试入口是 `lcqi_tests`。

Listing 2.1 第 2 章实践：张量布局、地址流与第一个 MatVec 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / linear_i8
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：手算一个 2 行 4 列权重矩阵的地址偏移。读者要先手写预期结果，再运行项目观察输出。动作是：把 `weight[out,k]` 展开成 `base+(out*K+k)*sizeof(i8)`，再对应到 `linear_i8` 的循环。如果输出里没有 `logit` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：张量不是裸数组；每个 kernel 入口都要能解释 dtype、shape、stride、scale 和输出缓冲这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一页地址流账本：输入向量、hidden 权重、输出 logits 各自的 shape、连续性、读写次数和错误边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第3章实践：GEMM、packing 与 SIMD 证据

本章对应原书中的第三册“从矩阵向量乘走向 GEMM、packing 与 SIMD”。不要把它当作概念复述，本章要把 packed weight、scalar baseline、AVX2 backend、output block、shape sweep、反汇编和降级放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `output_block`，核心命令或测试入口是 `lcqi_bench`。

Listing 3.1 第3章实践：GEMM、packing 与 SIMD 证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / output_block
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：同一组输入同时跑 `scalar`、`packed_scalar` 和 `avx2`。读者要先手写预期结果，再运行项目观察输出。动作是：比较 CSV 中 `backend`、`available`、`average_us`、`max_abs_diff` 和 `checksum`。如果输出里没有 `max_abs_diff` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：SIMD 路径只有在结果和 baseline 对齐后才有资格谈速度；`available=0` 也是证据，不是失败这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 shape sweep 表，至少解释一个 SIMD 有收益、一个收益不明显、一个当前环境不可验证的场景。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 4 章实践：int8 量化、累加器与重新量化

本章对应原书中的第三册“量化系统总览”和“int8 量化、累加器与重新量化”。不要把它当作概念复述，本章要把 int8 weight、float activation、scale、accumulator、requantize、oracle tolerance 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `TinyMlpModel`，核心命令或测试入口是 `lcqi_tests`。

Listing 4.1 第 4 章实践：int8 量化、累加器与重新量化的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<↵
↵ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / TinyMlpModel
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：构造一行 int8 权重 `[2, -1, 3, 0]` 和输入 `[1, 2, -1, 0.5]`。读者要先手写预期结果，再运行项目观察输出。动作是：先手算 int32/float accumulator，再和 `linear_i8` 输出比较。如果输出里没有 `scale` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：量化不是把 float 强转成 int8；必须记录 scale 作用位置、累加宽度、误差阈值和 golden 输出这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交量化合同卡：每个 tensor 的 dtype、scale 粒度、accumulator 类型、误差门限和拒绝条件。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

Transformer、KV Cache 与服务运行时

第 5 章实践：Transformer 切片与 KV Cache 状态

本章对应原书中的第三册“AI 推理原理、KV cache 与计算账本”和“KV cache 实现细节”。不要把它当作概念复述，本章要把 RMSNorm、Q/K/V、RoPE、GQA、KV slot、position、state edge 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `kv_cache`，核心命令或测试入口是 `lcqi_trace`。

Listing 5.1 第 5 章实践：Transformer 切片与 KV Cache 状态的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / kv_cache
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：固定 prompt `tok1tok2tok3`，观察第一个 decode step。读者要先手写预期结果，再运行项目观察输出。动作是：把 trace 中 tokenizer、q/k/v、`kv_cache_slot`、attention 和 logits 串成状态表。如果输出里没有 `token_position` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：KV cache 是请求状态，不是临时 activation；position、layer、head、layout 和生命周期必须进入 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 KV cache 状态表：每层每个 token 写入哪里、何时读取、何时禁止复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 6 章实践：Reference Decoder、logits 与采样

本章对应原书中的第三册“Reference decoder 实现”和“推理算法：logits、softmax、采样”。不要把它当作概念复述，本章要把 reference decoder、logits golden、argmax sampler、tokenizer contract hash、BOS/EOS 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 **argmax**，核心命令或测试入口是 **lcqi_trace**。

Listing 6.1 第 6 章实践：Reference Decoder、logits 与采样的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / argmax
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：让 trace 固定输出 tokenizer contract 和 logits golden。读者要先手写预期结果，再运行项目观察输出。动作是：比较 generated token、logits 数组和 tokenizer hash。如果输出里没有 **tokenizer_contract** 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：采样前必须先证明 logits 可复查；sampler 的随机性、温度和 top-k 只能建立在 reference 通过之后这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 **include/lcqi** 头文件和一个 **src** 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交采样报告：argmax、top-k、temperature 三种策略各自需要哪些额外字段才能复现。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 7 章实践：服务调度、SLO 与资源预算

本章对应原书中的第三册“业务服务实战：请求、调度、队列、取消与资源预算”。不要把它当作概念复述，本章要把 admission、KV budget、queue wait、TTFT、TPOT、cancel、reject、p95 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `ttft_ms`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 7.1 第 7 章实践：服务调度、SLO 与资源预算的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / ttft_ms
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：运行 serving probe 中短请求、RAG 请求、取消请求和超长请求。读者要先手写预期结果，再运行项目观察输出。动作是：解释 `memory_budget_exceeded`、cancel reclaim、`queue_wait_p95_ms` 和 `rejection_rate`。如果输出里没有 `rejection_rate` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：推理服务不是只看 tokens/s；是否接收请求、如何预留 KV、取消后是否回收，决定系统是否可运营这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 SLO 表：每类请求的 prompt/new token、预算、是否接受、TTFT、TPOT、拒绝原因和未验证边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 8 章实践：RAG、Agent 与状态图边界

本章对应原书中的第三册“上层 AI 应用编排：RAG、Agent、工具和状态图”。不要把它当作概念复述，本章要把 retrieval、tool call、prompt assembly、state graph、budget、trace id 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `req_rag`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 8.1 第 8 章实践：RAG、Agent 与状态图边界的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↳ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / req_rag
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 RAG 请求拆成 retrieve、rerank、prompt build、prefill、decode 五段。读者要先手写预期结果，再运行项目观察输出。动作是：为每段写输入输出和失败后能否重试。如果输出里没有 `prompt_tokens` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：上层编排不能把外部工具结果和模型状态混成字符串；每个边界都要有身份、预算和可重放记录这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 RAG 状态图：节点、边、超时、重试、缓存键、prompt 版本和审计字段。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

模型导入、AI 编译器与内存计划

第9章实践：模型导入、manifest 与 Shape Verifier

本章对应原书中的第三册“模型导入、manifest 与 shape verifier”。不要把它当作概念复述，本章要把 safetensors header、metadata、tensor name、dtype、shape、`data_offsets`、payload boundary 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `load_safetensors_manifest`，核心命令或测试入口是 `lcqi_tests`。

Listing 9.1 第9章实践：模型导入、manifest 与 Shape Verifier 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / load_safetensors_manifest
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：加载测试临时 safetensors fixture。读者要先手写预期结果，再运行项目观察输出。动作是：故意构造 offset 越界和 dtype/shape 字节数不匹配。如果输出里没有 `data_offsets` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型导入的第一职责是早拒绝：名字、shape、dtype、offset 和 payload 都必须在 kernel 运行前被验证这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 manifest gate 报告：合法 fixture 和两个坏 fixture 的字段、错误原因和拒绝位置。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 10 章实践：真实模型加载闭环与首 Token Trace

本章对应原书中的第三册“真实模型加载闭环：Tokenizer、权重格式与首 token trace”。不要把它当作概念复述，本章要把 tokenizer、weight mapping、chat template、first token trace、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

本章新增一个明确边界：tiny fixture 负责手算和逐层 oracle，真实 small 模型负责证明“开源模型资产确实能在本机 CPU 上加载并生成”。二者不能互相冒充。tiny MLP 算对，不等于真实 GGUF 加载器完成；真实 small 模型能输出文本，也不等于回答质量正确或 LCQI 自研后端已经支持完整 GGUF。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里有两组核心对象。第一组是 [tiny_tokenizer.txt](#)、reference decoder 和 [lcqi_trace](#)，用于手算 token 合同和逐 checkpoint 追踪。第二组是 [tools/run_smollm2_small_smoke.py](#)，用于下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，再通过 llama.cpp 在 CPU 上实际生成 assistant 文本。

Listing 10.1 第 10 章实践：真实模型加载闭环与首 Token Trace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / tiny_tokenizer.txt
make cpu3-smollm2-smoke
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：从 tokenizer fixture 开始，不跳到真实 7B。读者要先手写预期结果，再运行项目观察输出。动作是：解释 BOS/EOS、UNK、template hash 和 decoder 输入 tokens 的边界。如果输出里没有 `tokens=[0,1,2,3,4]` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：真实模型加载不是下载权重；必须先让 tokenizer 合同、权重 role、shape verifier 和首 token trace 形成闭环。这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来

本章的真实模型 smoke 固定为 [SmolLM2-135M-Instruct](#)，规模是 135M 参数，不是 tiny 随机夹具。脚本使用 GGUF 文件 [SmolLM2-135M-Instruct-Q4_K_M.gguf](#)，来源仓库是 [bartowski/SmolLM2-135M-Instruct-GGUF](#)，原模型组织是 [HuggingFaceTB](#)，license 字段为 [apache-2.0](#)。脚本不会把模型文件提交到仓库，而是缓存到用户目录；验收时检查文件大小 [105454432](#) 字节和 SHA256：

Listing 10.2 SmolLM2 small GGUF 文件身份

```
model_source_id=HuggingFaceTB/SmolLM2-135M-Instruct
```

```
model_gguf_repo_id=bartowski/SmolLM2-135M-Instruct-GGUF
model_file=SmolLM2-135M-Instruct-Q4_K_M.gguf
model_expected_bytes=105454432
model_expected_sha256=2↵
↵ e8040ceae7815abe0dc3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

运行命令是：

Listing 10.3 真实 small 模型 CPU 冒烟

```
make cpu3-smolllm2-smoke
```

这条命令会做四步。第一，下载或复用 GGUF 文件，并校验 SHA256，防止“文件名对了但内容不对”。第二，拉取固定 commit 的 `llama.cpp` 并构建 `llama-simple-chat`，避免依赖本机临时安装。第三，用 `-ngl0` 强制 CPU 路径，输入一个短 prompt，确认程序能走过 tokenizer、chat template、prefill、decode 和 sampler。第四，把报告写到：

Listing 10.4 真实模型 smoke 报告路径

```
books/cpu-volume-3/results/lcqi-smolllm2-small-model-smoke.txt
```

报告里必须至少出现这些字段：`model_expected_sha256`、`llama_cpp_commit`、`command`、`exit_code=0`、`validation=PASS` 和 `assistant_response`。如果只有下载日志，没有 assistant response，不能算通过；如果 assistant response 内容答错了，也不能改写成“质量通过”。本章只验证真实模型资产可加载、可 tokenize、可 decode、可生成非空文本。

四、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：运行真实 small 模型 smoke。报告不许只贴“命令执行成功”，必须解释脚本为什么校验模型 hash、为什么固定 `llama.cpp` commit、为什么使用 `-ngl0`，以及为什么 assistant response 非空只是 smoke，不是质量评测。

任务四：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。真实模型 smoke 的反例可以是：模型 SHA256 不匹配、网络不可用、`cmake` 缺失、assistant response 为空、命令超时。每个反例都要写清期望处理方式。

五、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。新增真实模型 smoke 后，最低验收还包括：模型身份有 hash，模型规模不是 tiny，CPU 路径实际运行，报告里有非空 assistant response，并明确声明这不是自研 LCQI 完整 GGUF 支持。

递进大作业：提交真实模型迁移清单：需要新增哪些 tokenizer、name mapping、quant descriptor 和 golden trace。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 11 章实践：运行时内存计划、liveness 与 Workspace

本章对应原书中的第三册“推理运行时与内存规划”。不要把它当作概念复述，本章要把 activation arena、workspace、liveness、state edge、debug dump、buffer reuse 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `layout`，核心命令或测试入口是 `lcqi_trace`。

Listing 11.1 第 11 章实践：运行时内存计划、liveness 与 Workspace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / layout
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：比较普通 activation 和 KV cache 的生命周期。读者要先手写预期结果，再运行项目观察输出。动作是：说明为什么 KV cache 不能被 activation planner 当作临时缓冲复用。如果输出里没有 `checksum` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：内存复用只在生命周期不重叠时成立；debug/oracle/materialization 会延长生命周期这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 liveness 表：每个 checkpoint 的 `defined_at`、`last_used_at`、是否 state、是否允许复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 12 章实践：IR、Pass、Lowering 与 Executable Plan

本章对应原书中的第三册“AI 编译器细化：IR、pass、lowering 与 executable plan”。不要把它当作概念复述，本章要把 typed graph、pass、fusion、layout propagation、backend capability、fallback reason 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `run_inference`，核心命令或测试入口是 `lcqi_cli`。

Listing 12.1 第 12 章实践：IR、Pass、Lowering 与 Executable Plan 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / run_inference
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 tiny MLP 写成三节点 IR：linear、relu、linear。读者要先手写预期结果，再运行项目观察输出。动作是：说明哪些 pass 可以融合，哪些必须保留为 oracle checkpoint。如果输出里没有 `backend` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：IR 的价值不是画图，而是让 shape、layout、量化、后端选择和 fallback 诊断有稳定承载物这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 executable plan 草图：节点、输入输出 tensor、layout、kernel、fallback reason 和 profiler event。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

后端优化、工业专项与验收

第 13 章实践：CPU 后端、微内核与性能证据

本章对应原书中的第三册“CPU 后端优化细讲”和“CPU decode 实战”。不要把它当作概念复述，本章要把 scalar baseline、packed weight、AVX2、objdump、perf boundary、shape sweep 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `avx2`，核心命令或测试入口是 `lcqi_bench`。

Listing 13.1 第 13 章实践：CPU 后端、微内核与性能证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↳ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / avx2
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：对同一 kernel 保存 benchmark CSV 和反汇编摘要。读者要先手写预期结果，再运行项目观察输出。动作是：解释速度差来自地址流、SIMD lane、packing、load 或 reduction 哪一项。如果输出里没有 `average_us` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：CPU 后端优化必须同时有 oracle、反汇编、shape sweep 和降级边界；只有耗时表不够这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 CPU kernel evidence：命令、机器、编译器、输入 shape、backend、checksum、反汇编摘要和未验证 PMU 边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 14 章实践：GPU 后端边界、copy 与同步成本

本章对应原书中的第三册“GPU 硬件基础”和“GPU 后端优化细讲”。不要把它当作概念复述，本章要把 device buffer、stream、event、kernel launch、copy、sync、fallback 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 **StepTrace**，核心命令或测试入口是 **lcqi_trace**。

Listing 14.1 第 14 章实践：GPU 后端边界、copy 与同步成本的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / StepTrace
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：当前项目没有 GPU kernel，也要写清 adapter 合同。读者要先手写预期结果，再运行项目观察输出。动作是：把 CPU trace 字段扩展成 GPU StepTrace 需要哪些字段。如果输出里没有 **copy_time_ns** 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：GPU 快不快不能只看 kernel；copy、sync、sampler、fallback 和 batch wait 必须进入端到端 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 **include/lcqi** 头文件和一个 **src** 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 GPU adapter 设计：buffer owner、stream、event、H2D/D2H 字节、sync 点、fallback 和 oracle 对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 15 章实践：工业 LLM 专项能力对照

本章对应原书中的第三册“工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照”。不要把它当作概念复述，本章要把 speculative decoding、MoE routing、LoRA adapter、multi-GPU、source reading 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `bandwidth_limit`，核心命令或测试入口是 `lcqi_ledger`。

Listing 15.1 第 15 章实践：工业 LLM 专项能力对照的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<↵
↵ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_ledger / bandwidth_limit
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：选择一个专项，不写泛泛介绍。读者要先手写预期结果，再运行项目观察输出。动作是：把它接到现有 trace、memory、backend 或 serving 字段上。如果输出里没有 `tokens/s` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：工业专项不是功能清单；必须说明它购买什么能力、增加什么状态、破坏哪些假设这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交专项设计卡：动机、状态新增、trace 字段、回归测试、性能账本和不能验证的部分。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 16 章实践：工程验收、回归门禁与最终项目

本章对应原书中的第三册“工程验收与回归体系”。不要把它当作概念复述，本章要把 reference、unit test、trace golden、benchmark gate、SLO gate、coverage、CI 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `lcqi_tests`，核心命令或测试入口是 `ctest`。

Listing 16.1 第 16 章实践：工程验收、回归门禁与最终项目的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
↳ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: ctest / lcqi_tests
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把所有前面报告合成最终验收包。读者要先手写预期结果，再运行项目观察输出。动作是：运行 `ctest` 并说明每个 test 保护哪条合同。如果输出里没有 `100% tests passed` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：实践卷最终目标是长期维护：结果正确、trace 稳定、性能可解释、服务指标可回归、失败边界不伪装这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交最终项目：README、命令记录、报告索引、测试结果、benchmark 表、trace 摘要、风险清单和下一阶段计划。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

完整源码清单

本附录从 `books/cpu-volume-3/labs/linux_cpu_inference` 直接读入贯穿项目源码。实践卷正文只解释每章要看哪一段；真正的闭环必须回到完整工程：头文件、实现、CLI、benchmark、trace、测试和模型 fixture 都要能一起构建。

一、构建入口和项目说明

Listing 16.2 第三册 CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)
10
11 if(CPU_VOLUME_3_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/linux_cpu_inference)
14 endif()
```

Listing 16.3 linux_cpu_inference CMakeLists.txt

```
1 add_library(lcqi STATIC
2     src/inference.cpp
3     src/int8_kernels.cpp
4     src/model.cpp
5     src/reference_decoder.cpp
6     src/safetensors.cpp
7     src/tokenizer.cpp
8 )
9
10 if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
11     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
12         target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
13         set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↵
14             ↵ COMPILE_OPTIONS "-mavx2;-mfma")
15         target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
16     endif()
17 endif()
18
19 if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
20     message(WARNING "LCQI is written for the book's local Linux CPU target; ↵
21         ↵ current system is ${CMAKE_SYSTEM_NAME}.")
22 endif()
```



```
21
22 target_include_directories(lcqi
23     PUBLIC
24     ${CMAKE_CURRENT_SOURCE_DIR}/include
25 )
26
27 target_compile_features(lcqi PUBLIC cxx_std_20)
28
29 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
30     target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
31 endif()
32
33 add_executable(lcqi_cli
34     src/main.cpp
35 )
36
37 target_link_libraries(lcqi_cli PRIVATE lcqi)
38
39 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
40     target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
41 endif()
42
43 add_executable(lcqi_bench
44     src/bench.cpp
45 )
46
47 target_link_libraries(lcqi_bench PRIVATE lcqi)
48
49 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
50     target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
51 endif()
52
53 add_executable(lcqi_trace
54     src/trace.cpp
55 )
56
57 target_link_libraries(lcqi_trace PRIVATE lcqi)
58
59 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
60     target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
61 endif()
62
63 add_executable(lcqi_ledger
64     src/ledger.cpp
65 )
66
67 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
68     target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
69 endif()
70
71 add_executable(lcqi_serving_probe
72     src/serving_probe.cpp
```

```

73 )
74
75 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
76     target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
77 endif()
78
79 find_package(dnnl QUIET)
80 if(dnnl_FOUND)
81     add_executable(lcqi_onednn_bench
82         src/onednn_bench.cpp
83     )
84     target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
85     target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
86     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
87         target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -↵
↵ Wpedantic)
88     endif()
89 endif()
90
91 add_executable(lcqi_tests
92     tests/lcqi_tests.cpp
93 )
94
95 target_link_libraries(lcqi_tests PRIVATE lcqi)
96
97 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
98     target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
99 endif()
100
101 add_test(NAME lcqi_tests COMMAND ${<TARGET_FILE:lcqi_tests>})
102 add_test(NAME lcqi_trace COMMAND ${<TARGET_FILE:lcqi_trace>})
103 set_tests_properties(lcqi_trace PROPERTIES
104     PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
105 )
106 add_test(NAME lcqi_ledger COMMAND ${<TARGET_FILE:lcqi_ledger>})
107 set_tests_properties(lcqi_ledger PROPERTIES
108     PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
109 )
110 add_test(NAME lcqi_serving_probe COMMAND ${<TARGET_FILE:lcqi_serving_probe>})
111 set_tests_properties(lcqi_serving_probe PROPERTIES
112     PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
113 )

```

Listing 16.4 linux_cpu_inference README.md

```

1 # Linux CPU Quantized Inference
2
3 这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三↵
↵ 册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大↵
↵ 模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache↵
↵ 、CPU/GPU 后端、正确性报告和性能对标。
4

```

```

5 目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器
  ↳ 版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线
  ↳ 程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源
  ↳ 码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变
  ↳ 当前切片的语义合同。

6
7 当前版本支持：
8
9 - 教学文本模型格式 `LCQI_MODEL_V1`
10 - 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
11 - safetensors header manifest 解析：metadata、tensor name、dtype、shape、data
  ↳ offsets、offset 边界和 dtype/shape 字节数校验
12 - 两层 MLP 教学切片
13 - int8 权重加 per-layer scale
14 - float 输入和激活
15 - 量化线性层、ReLU、分类 argmax
16 - int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
17 - tiny reference decoder: RMSNorm、float linear、RoPE、GQA KV cache、decode
  ↳ attention、SwiGLU、lm head
18 - reference trace CSV: 从 prompt/tokenizer 合同到 logits/sampler/token，逐
  ↳ checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values
  ↳ 摘要
19 - 7B ledger CSV: 输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth
  ↳ only tokens/s 上限
20 - serving SLO probe CSV: 输出 admission、batch state、KV 预算、queue wait、TTFT
  ↳ 、TPOT、取消回收和拒绝率
21 - 真实 small 开源模型冒烟：通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量
  ↳ 化文件，在本地 CPU 上生成一段 assistant 文本，并把模型 hash、命令、输出和
  ↳ 性能行写入报告
22 - CLI 推理和简单 benchmark
23 - CTest 正确性测试
24
25 后续扩展方向：
26
27 - 真实 7B 级模型 metadata 和 tokenizer
28 - 完整 BPE/SentencePiece tokenizer、GGUF/safetensors 权重导入、分片加载和 name
  ↳ mapping
29 - 多层 decoder-only Transformer reference path
30 - KV cache 分页、内存规划和可选量化
31 - CPU packed weight、SIMD、线程和 NUMA 优化
32 - GPU backend adapter 和 device kernel
33 - 与现有框架的 prefill/decode/内存/误差对标报告
34
35 构建：
36
37 ```bash
38 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -
  ↳ DCMMAKE_BUILD_TYPE=Debug
39 cmake --build books/cpu-volume-3/build/lcqi-debug
40 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
41 ```
42

```

```

43 运行：
44
45 ```bash
46 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
47   books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
48   1.0 2.0 -1.0 0.5 1000
49 ```
50
51 kernel benchmark:
52
53 ```bash
54 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
55 ```
56
57 输出 CSV 字段：
58
59 ```text
60 target_arch,input_size,output_size,output_block,repeat,backend,available,↵
61   ↵ average_us,max_abs_diff,checksum
62 ```
63
64 当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 ↵
65   ↵ 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `↵
66   ↵ available=0`，避免把源码存在误报成当前环境实测性能。
67
68 这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout↵
69   ↵ 、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继↵
70   ↵ 续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对↵
71   ↵ 照、sanitizer、fuzz 和 CI。
72
73 safetensors manifest gate:
74
75 `lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、`↵
76   ↵ __metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 ↵
77   ↵ dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape ↵
78   ↵ 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，↵
79   ↵ 不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模↵
80   ↵ 型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant ↵
81   ↵ descriptor 和 first token trace。
82
83 reference decoder trace:
84
85 ```bash
86 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
87 ```
88
89 输出 CSV 字段：
90
91 ```text
92 name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
93 ```

```

```

83 这条 trace 固定 prompt fixture `tok1 tok2 tok3`, tokenizer 输出完整 `tokens`
    ↪ `=[0,1,2,3,4]`, 其中 `0/4` 是 BOS/EOS; decoder trace 会剥离 BOS/EOS 后进入
    ↪ tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同, 避免
    ↪ 把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/
    ↪ K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、
    ↪ logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`
    ↪ lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、
    ↪ stride、dtype、layout 和 logits golden, CTest 也会直接运行 `lcqi_trace`。

84
85 当前 tokenizer fixture 的关键输出:
86
87 ```text
88 tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
89 tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-
    ↪ chat-template-v1
90 ```
91
92 7B ledger:
93
94 ```bash
95 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
96   > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
97 ```
98
99 输出 CSV 字段:
100
101 ```text
102 section,item,value,unit,notes
103 ```
104
105 这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`, 报告 decode
    ↪ FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s
    ↪ 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行, 防止账本公
    ↪ 式漂移。

106
107 serving SLO probe:
108
109 ```bash
110 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe
    ↪ \
111   > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
112 ```
113
114 输出 CSV 字段:
115
116 ```text
117 row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,
    ↪ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,
    ↪ tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,
    ↪ metric_value
118 ```
119

```

```

120 probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按
    ↪ `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回
    ↪ `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`
    ↪ rejection_rate` 等服务化指标。
121
122 真实 small 开源模型 smoke：
123
124 ```bash
125 make cpu3-smollm2-smoke
126 ```
127
128 这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型
    ↪ 文件缓存到 `~/.cache/lcqi-smollm2-small-smoke`，模型不进入 Git。固定模型
    ↪ 是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `
    ↪ bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-
    ↪ Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256：
129
130 ```text
131 2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
132 ```
133
134 脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短
    ↪ prompt，检查 assistant response 非空，并生成：
135
136 ```text
137 books/cpu-volume-3/results/lcqi-smollm2-small-model-smoke.txt
138 ```
139
140 这份报告只能证明真实 GGUF small 模型已经完成加载、tokenizer/chat template、
    ↪ prefill、decode 和文本输出。它不能证明 LCQI 自研 C++ 引擎已经支持完整
    ↪ GGUF、不能证明回答质量正确，也不能替代 tiny fixture 的逐层 golden trace。
    ↪ tiny fixture 继续负责可手算语义和回归定位；SmolLM2 smoke 负责把“真实开源
    ↪ 模型能在本机跑起来”变成可复查证据。

```

Listing 16.5 tools/run_smollm2_small_smoke.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import hashlib
7  import os
8  import re
9  import shlex
10 import shutil
11 import subprocess
12 import sys
13 from dataclasses import dataclass
14 from pathlib import Path
15
16

```



```

17 MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
18 MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
19 MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
20 MODEL_URL = (
21     "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
22     f"/resolve/main/{MODEL_FILE_NAME}"
23 )
24 MODEL_EXPECTED_BYTES = 105454432
25 MODEL_EXPECTED_SHA256 = (
26     "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
27 )
28 LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
29 LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
30 LLAMA_TARGET = "llama-simple-chat"
31 DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
32 DEFAULT_CONTEXT_TOKENS = 512
33 DEFAULT_TIMEOUT_SECONDS = 180
34 DEFAULT_BUILD_JOBS = 2
35
36
37 @dataclass(frozen=True)
38 class CommandResult:
39     args: list[str]
40     returncode: int
41     stdout: str
42     stderr: str
43
44
45 def script_path() -> Path:
46     return Path(__file__).resolve()
47
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def repo_dir() -> Path:
54     return script_path().parents[3]
55
56
57 def default_cache_dir() -> Path:
58     explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
59     if explicit_cache:
60         return Path(explicit_cache).expanduser()
61     xdg_cache = os.environ.get("XDG_CACHE_HOME")
62     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "←
    ↪ .cache"
63     return cache_root / "lcqi-smolllm2-small-smoke"
64
65
66 def default_report_path() -> Path:
67     return volume_dir() / "results" / "lcqi-smolllm2-small-model-smoke.txt"

```

```

68
69
70 def command_line(args: list[str]) -> str:
71     return " ".join(shlex.quote(arg) for arg in args)
72
73
74 def require_tool(name: str) -> str:
75     path = shutil.which(name)
76     if path is None:
77         raise RuntimeError(f"required tool not found in PATH: {name}")
78     return path
79
80
81 def run_command(
82     args: list[str],
83     *,
84     cwd: Path | None = None,
85     input_text: str | None = None,
86     timeout_seconds: int | None = None,
87 ) -> CommandResult:
88     try:
89         completed = subprocess.run(
90             args,
91             cwd=str(cwd) if cwd else None,
92             input=input_text,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=
↵ stderr)
102
103     return CommandResult(
104         args=args,
105         returncode=completed.returncode,
106         stdout=completed.stdout,
107         stderr=completed.stderr,
108     )
109
110
111 def ensure_success(result: CommandResult, action: str) -> None:
112     if result.returncode == 0:
113         return
114     raise RuntimeError(
115         f"{action} failed with exit code {result.returncode}\n"
116         f"command: {command_line(result.args)}\n"
117         f"stdout tail:\n{tail(result.stdout)}\n"
118         f"stderr tail:\n{tail(result.stderr)}"

```

```

119     )
120
121
122 def tail(text: str, max_chars: int = 4000) -> str:
123     if len(text) <= max_chars:
124         return text
125     return text[-max_chars:]
126
127
128 def sha256_file(path: Path) -> str:
129     digest = hashlib.sha256()
130     with path.open("rb") as handle:
131         for block in iter(lambda: handle.read(1024 * 1024), b''):
132             digest.update(block)
133     return digest.hexdigest()
134
135
136 def verify_model_file(path: Path) -> tuple[bool, str, int]:
137     if not path.exists():
138         return False, "", 0
139     actual_bytes = path.stat().st_size
140     actual_sha = sha256_file(path)
141     return (
142         actual_bytes == MODEL_EXPECTED_BYTES
143         and actual_sha == MODEL_EXPECTED_SHA256,
144         actual_sha,
145         actual_bytes,
146     )
147
148
149 def download_model(model_path: Path, *, force_download: bool, no_download: bool ↵
    ↵ ) -> None:
150     model_path.parent.mkdir(parents=True, exist_ok=True)
151
152     if force_download and model_path.exists():
153         model_path.unlink()
154
155     is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
156     if is_valid:
157         print(f"[lcqi-smoke] model cache hit: {model_path}")
158         return
159
160     if model_path.exists():
161         print(
162             "[lcqi-smoke] cached model hash mismatch, redownloading: "
163             f"bytes={actual_bytes}, sha256={actual_sha}"
164         )
165         model_path.unlink()
166
167     if no_download:
168         raise RuntimeError(
169             f"model file is missing or invalid and --no-download was set: {↵

```

```

170     ↪ model_path}"
171     )
172     require_tool("curl")
173     part_path = model_path.with_suffix(model_path.suffix + ".part")
174     if part_path.exists():
175         part_path.unlink()
176
177     print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{↪
178     ↪ MODEL_FILE_NAME}")
179     result = run_command(
180         [
181             "curl",
182             "-L",
183             "--fail",
184             "--retry",
185             "3",
186             "-o",
187             str(part_path),
188             MODEL_URL,
189         ]
190     )
191     ensure_success(result, "download SmolLM2 GGUF")
192
193     downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(↪
194     ↪ part_path)
195     if not downloaded_ok:
196         raise RuntimeError(
197             "downloaded model did not match expected identity: "
198             f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
199         )
200     part_path.replace(model_path)
201     print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")
202
203 def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) ↪
204     ↪ -> Path:
205     cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
206     source_dir = cache_dir / "llama.cpp"
207     build_dir = source_dir / "build"
208     if cached_binary.exists() and os.access(cached_binary, os.X_OK) and ↪
209     ↪ is_pinned_llama_checkout(source_dir):
210         print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
211         return cached_binary
212
213     if skip_build:
214         raise RuntimeError(f"llama.cpp binary is missing and --skip-build was ↪
215         ↪ set: {cached_binary}")
216
217     require_tool("git")
218     require_tool("cmake")

```

```

216     cache_dir.mkdir(parents=True, exist_ok=True)
217
218     if not (source_dir / ".git").exists():
219         print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
220         result = run_command(
221             [
222                 "git",
223                 "clone",
224                 "--filter=blob:none",
225                 "--no-checkout",
226                 LLAMA_CPP_REPO_URL,
227                 str(source_dir),
228             ]
229         )
230         ensure_success(result, "clone llama.cpp")
231
232     print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {↵
↵ LLAMA_CPP_COMMIT}")
233     fetch = run_command(
234         ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", ↵
↵ LLAMA_CPP_COMMIT]
235     )
236     ensure_success(fetch, "fetch pinned llama.cpp commit")
237     checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach↵
↵ ", LLAMA_CPP_COMMIT])
238     ensure_success(checkout, "checkout pinned llama.cpp commit")
239
240     print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
241     configure = run_command(
242         [
243             "cmake",
244             "-S",
245             str(source_dir),
246             "-B",
247             str(build_dir),
248             "-DCMAKE_BUILD_TYPE=Release",
249             "-DLLAMA_BUILD_TESTS=OFF",
250             "-DLLAMA_BUILD_EXAMPLES=ON",
251             "-DLLAMA_BUILD_SERVER=OFF",
252             "-DGGML_NATIVE=OFF",
253         ]
254     )
255     ensure_success(configure, "configure llama.cpp")
256
257     print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
258     build = run_command(
259         ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", ↵
↵ str(jobs)]
260     )
261     ensure_success(build, f"build {LLAMA_TARGET}")
262
263     if not cached_binary.exists():

```

```

264     raise RuntimeError(f"expected binary was not produced: {cached_binary}" ←
    ↪ )
265     return cached_binary
266
267
268 def is_pinned_llama_checkout(source_dir: Path) -> bool:
269     if not (source_dir / ".git").exists():
270         return False
271     result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
272     if result.returncode != 0:
273         return False
274     return result.stdout.strip() == LLAMA_CPP_COMMIT
275
276
277 def strip_ansi(text: str) -> str:
278     return re.sub(r"\x1b\[([0-?]*[ -/]*[@-~])", "", text)
279
280
281 def extract_response(stdout: str) -> str:
282     cleaned = strip_ansi(stdout).replace("\r", "")
283     chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if ←
    ↪ chunk.strip()]
284     if not chunks:
285         return ""
286     return chunks[0]
287
288
289 def run_model_smoke(
290     llama_binary: Path,
291     model_path: Path,
292     *,
293     prompt: str,
294     context_tokens: int,
295     timeout_seconds: int,
296 ) -> tuple[CommandResult, str]:
297     command = [
298         str(llama_binary),
299         "-m",
300         str(model_path),
301         "-ngl",
302         "0",
303         "-c",
304         str(context_tokens),
305     ]
306     print("[lcqi-smoke] running real small model generation")
307     result = run_command(
308         command,
309         input_text=prompt.rstrip("\n") + "\n\n",
310         timeout_seconds=timeout_seconds,
311     )
312     ensure_success(result, "run small model smoke")
313

```



```

314     response = extract_response(result.stdout)
315     if not response:
316         raise RuntimeError(
317             "small model smoke exited successfully but produced no assistant ←
↪ text\n"
318             f"stdout:\n{result.stdout}\n"
319             f"stderr:\n{result.stderr}"
320         )
321     return result, response
322
323
324 def current_repo_commit() -> str:
325     result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short", ←
↪ "HEAD"])
326     if result.returncode != 0:
327         return "unknown"
328     return result.stdout.strip()
329
330
331 def current_uname() -> str:
332     result = run_command(["uname", "-a"])
333     if result.returncode != 0:
334         return "unknown"
335     return result.stdout.strip()
336
337
338 def write_report(
339     report_path: Path,
340     *,
341     cache_dir: Path,
342     llama_binary: Path,
343     model_path: Path,
344     prompt: str,
345     result: CommandResult,
346     response: str,
347 ) -> None:
348     report_path.parent.mkdir(parents=True, exist_ok=True)
349     clean_stdout = strip_ansi(result.stdout).strip()
350     report = "\n".join(
351         [
352             "LCQI SmolLM2 Small Model Smoke",
353             "",
354             f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
355             f"repo_commit={current_repo_commit()}",
356             f"host={current_uname()}",
357             f"python={sys.version.split()[0]}",
358             f"cache_dir={cache_dir}",
359             "",
360             f"model_source_id={MODEL_SOURCE_ID}",
361             f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
362             f"model_file={MODEL_FILE_NAME}",
363             f"model_url={MODEL_URL}",

```

```

364         f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
365         f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
366         f"model_path={model_path}",
367         "",
368         f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
369         f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
370         f"llama_target={LLAMA_TARGET}",
371         f"llama_binary={llama_binary}",
372         "",
373         f"prompt={prompt}",
374         f"command={command_line(result.args)}",
375         f"exit_code={result.returncode}",
376         "",
377         "validation=PASS: model hash matched, llama-simple-chat exited 0, "
378         "assistant response was non-empty",
379         "",
380         "assistant_response:",
381         response,
382         "",
383         "stdout_clean_tail:",
384         tail(clean_stdout, 2000),
385         "",
386         "stderr_tail:",
387         tail(result.stderr.strip(), 2000),
388         "",
389     ]
390 )
391 report_path.write_text(report, encoding="utf-8")
392
393
394 def parse_args() -> argparse.Namespace:
395     parser = argparse.ArgumentParser(
396         description=(
397             "Run a real local small open-source model smoke for CPU Volume 3. "
398             "The script caches llama.cpp and the GGUF model outside the Git ↵
399         ↵ repository."
400     )
401     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
402     parser.add_argument("--model-path", type=Path, default=None)
403     parser.add_argument("--llama-bin", type=Path, default=None)
404     parser.add_argument("--report", type=Path, default=default_report_path())
405     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
406     parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
407     parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
408     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
409     parser.add_argument("--force-download", action="store_true")
410     parser.add_argument("--no-download", action="store_true")
411     parser.add_argument("--skip-build", action="store_true")
412     return parser.parse_args()
413
414

```

```

415 def main() -> int:
416     args = parse_args()
417     cache_dir = args.cache_dir.expanduser().resolve()
418     model_path = (
419         args.model_path.expanduser().resolve()
420         if args.model_path
421         else cache_dir / "models" / MODEL_FILE_NAME
422     )
423     report_path = args.report.expanduser().resolve()
424
425     try:
426         download_model(
427             model_path,
428             force_download=args.force_download,
429             no_download=args.no_download,
430         )
431         if args.llama_bin:
432             llama_binary = args.llama_bin.expanduser().resolve()
433             if not llama_binary.exists() or not os.access(llama_binary, os.X_OK):
434                 raise RuntimeError(f"--llama-bin is not executable: {llama_binary}")
435         else:
436             llama_binary = ensure_llama_simple_chat(
437                 cache_dir,
438                 jobs=args.jobs,
439                 skip_build=args.skip_build,
440             )
441
442         result, response = run_model_smoke(
443             llama_binary,
444             model_path,
445             prompt=args.prompt,
446             context_tokens=args.ctx,
447             timeout_seconds=args.timeout,
448         )
449         write_report(
450             report_path,
451             cache_dir=cache_dir,
452             llama_binary=llama_binary,
453             model_path=model_path,
454             prompt=args.prompt,
455             result=result,
456             response=response,
457         )
458     except RuntimeError as exc:
459         print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
460         return 1
461
462     print(f"report={report_path}")
463     print(f"model_sha256={MODEL_EXPECTED_SHA256}")
464     print(f"assistant_response={response}")

```

```

465     return 0
466
467
468 if __name__ == "__main__":
469     raise SystemExit(main())

```

二、公共合同头文件

Listing 16.6 include/lcqi/inference.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <span>
5  #include <vector>
6
7  #include <lcqi/model.hpp>
8
9  namespace lcqi {
10
11  struct InferenceResult {
12      std::vector<float> logits;
13      std::int32_t predicted_class = 0;
14  };
15
16  void linear_i8(const QuantizedLinearLayer& layer,
17               std::span<const float> input,
18               std::span<float> output);
19
20  void relu_inplace(std::span<float> values);
21
22  InferenceResult run_inference(const TinyMlpModel& model,
23                               std::span<const float> input);
24
25 } // namespace lcqi

```

Listing 16.7 include/lcqi/int8_kernels.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <span>
5  #include <vector>
6
7  #include <lcqi/model.hpp>
8
9  namespace lcqi {
10
11  struct PackedLinearI8 {
12      std::int32_t input_size = 0;
13      std::int32_t output_size = 0;

```

```

14     std::int32_t output_block_size = 0;
15     float scale = 1.0F;
16     std::vector<std::int8_t> packed_weights;
17     std::vector<float> bias;
18 };
19
20 struct KernelBenchmarkCase {
21     std::int32_t input_size = 0;
22     std::int32_t output_size = 0;
23     std::int32_t output_block_size = 0;
24     std::int32_t repeat = 0;
25 };
26
27 struct KernelBackendBenchmark {
28     const char* name = "";
29     bool available = false;
30     double average_us = 0.0;
31     float max_abs_diff = 0.0F;
32     float checksum = 0.0F;
33 };
34
35 struct KernelBenchmarkResult {
36     KernelBenchmarkCase benchmark_case;
37     std::vector<KernelBackendBenchmark> backends;
38 };
39
40 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
41                               std::int32_t output_block_size);
42
43 void linear_i8_packed(const PackedLinearI8& layer,
44                      std::span<const float> input,
45                      std::span<float> output);
46
47 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
48                                      std::span<const float> input,
49                                      std::span<float> output,
50                                      std::span<float> accumulator_workspace);
51
52 bool linear_i8_packed_avx2_available() noexcept;
53
54 void linear_i8_packed_avx2(const PackedLinearI8& layer,
55                            std::span<const float> input,
56                            std::span<float> output);
57
58 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
59                                                  std::int32_t output_size,
60                                                  float scale);
61
62 std::vector<float> make_deterministic_input(std::int32_t input_size);
63
64 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);
65

```

```

66 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ←
    ↪ benchmark_case);
67
68 } // namespace lcqi

```

Listing 16.8 include/lcqi/model.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct QuantizedLinearLayer {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     float scale = 1.0F;
14     std::vector<std::int8_t> weights;
15     std::vector<float> bias;
16 };
17
18 struct TinyMlpModel {
19     std::int32_t input_size = 0;
20     std::int32_t hidden_size = 0;
21     std::int32_t output_size = 0;
22     QuantizedLinearLayer hidden;
23     QuantizedLinearLayer output;
24 };
25
26 TinyMlpModel load_model(const std::filesystem::path& path);
27
28 } // namespace lcqi

```

Listing 16.9 include/lcqi/reference_decoder.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct LinearWeightsF32 {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     std::vector<float> weights;

```

```

14     std::vector<float> bias;
15 };
16
17 struct DecoderConfig {
18     std::int32_t hidden_size = 0;
19     std::int32_t query_heads = 0;
20     std::int32_t kv_heads = 0;
21     std::int32_t head_dim = 0;
22     std::int32_t intermediate_size = 0;
23     std::int32_t vocab_size = 0;
24     std::int32_t max_context = 0;
25     float rms_epsilon = 1.0e-5F;
26     float rope_base = 10000.0F;
27 };
28
29 struct DecoderLayerWeightsF32 {
30     std::vector<float> rms_attention_weight;
31     LinearWeightsF32 wq;
32     LinearWeightsF32 wk;
33     LinearWeightsF32 wv;
34     LinearWeightsF32 wo;
35     std::vector<float> rms_mlp_weight;
36     LinearWeightsF32 w_gate;
37     LinearWeightsF32 w_up;
38     LinearWeightsF32 w_down;
39 };
40
41 struct ReferenceDecoderModel {
42     DecoderConfig config;
43     std::vector<float> token_embedding;
44     std::vector<DecoderLayerWeightsF32> layers;
45     std::vector<float> final_norm_weight;
46     LinearWeightsF32 lm_head;
47 };
48
49 struct ReferenceDecodeResult {
50     std::vector<float> logits;
51     std::int32_t predicted_token = 0;
52 };
53
54 struct ReferenceTensorCheckpoint {
55     std::string name;
56     std::int32_t token_position = 0;
57     std::int32_t layer_id = 0;
58     std::vector<std::int32_t> shape;
59     std::vector<std::int32_t> stride;
60     std::string dtype;
61     std::string layout;
62     float checksum = 0.0F;
63     float max_abs = 0.0F;
64     std::vector<float> values;
65 };

```



```

66
67 struct ReferenceDecodeTraceResult {
68     ReferenceDecodeResult result;
69     std::vector<ReferenceTensorCheckpoint> checkpoints;
70 };
71
72 class ReferenceKVCache {
73 public:
74     ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);
75
76     void append(std::int32_t layer_id,
77                 std::int32_t model_position,
78                 std::span<const float> key,
79                 std::span<const float> value);
80
81     [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
82                                              std::int32_t model_position,
83                                              std::int32_t kv_head) const;
84
85     [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
86                                                std::int32_t model_position,
87                                                std::int32_t kv_head) const;
88
89     [[nodiscard]] std::int32_t layer_count() const noexcept;
90     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
91
92 private:
93     [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
94                                           std::int32_t model_position,
95                                           std::int32_t kv_head) const;
96     void validate_address(std::int32_t layer_id,
97                           std::int32_t model_position,
98                           std::int32_t kv_head) const;
99
100     DecoderConfig config_;
101     std::int32_t layer_count_ = 0;
102     std::int32_t filled_tokens_ = 0;
103     std::vector<float> keys_;
104     std::vector<float> values_;
105     std::vector<std::uint8_t> written_;
106 };
107
108 void rms_norm(std::span<const float> input,
109               std::span<const float> weight,
110               float epsilon,
111               std::span<float> output);
112
113 void linear_f32(const LinearWeightsF32& layer,
114                 std::span<const float> input,
115                 std::span<float> output);
116
117 void apply_rope(std::span<float> heads,

```

```

118         std::int32_t head_count,
119         std::int32_t head_dim,
120         std::int32_t model_position,
121         float rope_base);
122
123 void attention_decode(const DecoderConfig& config,
124                     const ReferenceKVCache& cache,
125                     std::int32_t layer_id,
126                     std::int32_t model_position,
127                     std::span<const float> query,
128                     std::span<float> output);
129
130 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
131                                         std::span<const std::int32_t> ↵
132                                         ↵ token_ids);
133
134 ReferenceDecodeTraceResult run_reference_decode_with_trace(
135     const ReferenceDecoderModel& model,
136     std::span<const std::int32_t> token_ids);
137
138 ReferenceDecoderModel make_tiny_reference_decoder_model();
139 } // namespace lcqi

```

Listing 16.10 include/lcqi/safetensors.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <utility>
8 #include <vector>
9
10 namespace lcqi {
11
12 struct SafeTensorEntry {
13     std::string name;
14     std::string dtype;
15     std::vector<std::int64_t> shape;
16     std::uint64_t data_begin = 0;
17     std::uint64_t data_end = 0;
18
19     [[nodiscard]] std::uint64_t byte_size() const;
20 };
21
22 struct SafeTensorManifest {
23     std::uint64_t header_size = 0;
24     std::uint64_t data_start_offset = 0;
25     std::vector<std::pair<std::string, std::string>> metadata;
26     std::vector<SafeTensorEntry> tensors;

```

```

27
28     [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) ↵
    ↵ const;
29 };
30
31 [[nodiscard]] SafeTensorManifest load_safetensors_manifest(
32     const std::filesystem::path& path);
33
34 } // namespace lcqi

```

Listing 16.11 include/lcqi/tokenizer.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <vector>
8
9 namespace lcqi {
10
11 struct TokenEntry {
12     std::string text;
13     std::int32_t id = 0;
14 };
15
16 struct TokenizerModel {
17     std::int32_t bos_id = -1;
18     std::int32_t eos_id = -1;
19     std::int32_t unk_id = -1;
20     std::string chat_template_hash;
21     std::vector<TokenEntry> vocab;
22 };
23
24 [[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);
25
26 [[nodiscard]] std::vector<std::int32_t> encode_prompt(
27     const TokenizerModel& tokenizer,
28     std::string_view prompt);
29
30 [[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↵
    ↵ tokenizer);
31
32 } // namespace lcqi

```

三、核心实现

Listing 16.12 src/inference.cpp

```

1 #include <lcqi/inference.hpp>

```

```

2
3 #include <algorithm>
4 #include <cmath>
5 #include <stdexcept>
6
7 namespace lcqi {
8 namespace {
9
10 void validate_input_size(std::span<const float> input, std::int32_t expected) {
11     if (input.size() != static_cast<std::size_t>(expected)) {
12         throw std::runtime_error("input size does not match model shape");
13     }
14 }
15
16 std::int32_t argmax(std::span<const float> values) {
17     if (values.empty()) {
18         throw std::runtime_error("cannot take argmax of empty logits");
19     }
20     std::int32_t best_index = 0;
21     float best_value = values[0];
22     for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i)↵
    ↵ {
23         const float candidate = values[static_cast<std::size_t>(i)];
24         if (candidate > best_value) {
25             best_value = candidate;
26             best_index = i;
27         }
28     }
29     return best_index;
30 }
31
32 } // namespace
33
34 void linear_i8(const QuantizedLinearLayer& layer,
35               std::span<const float> input,
36               std::span<float> output) {
37     if (input.size() != static_cast<std::size_t>(layer.input_size)) {
38         throw std::runtime_error("linear_i8 input size mismatch");
39     }
40     if (output.size() != static_cast<std::size_t>(layer.output_size)) {
41         throw std::runtime_error("linear_i8 output size mismatch");
42     }
43
44     for (std::int32_t out = 0; out < layer.output_size; ++out) {
45         const std::size_t row_base =
46             static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.↵
    ↵ input_size);
47         float acc = layer.bias[static_cast<std::size_t>(out)];
48         for (std::int32_t in = 0; in < layer.input_size; ++in) {
49             const std::int8_t weight =
50                 layer.weights[row_base + static_cast<std::size_t>(in)];
51             acc += static_cast<float>(weight) * layer.scale *

```

```

52         input[static_cast<std::size_t>(in)];
53     }
54     output[static_cast<std::size_t>(out)] = acc;
55 }
56 }
57
58 void relu_inplace(std::span<float> values) {
59     for (float& value : values) {
60         value = std::max(0.0F, value);
61     }
62 }
63
64 InferenceResult run_inference(const TinyMlpModel& model,
65                               std::span<const float> input) {
66     validate_input_size(input, model.input_size);
67
68     std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F↵
↵ );
69     std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F↵
↵ );
70
71     linear_i8(model.hidden, input, hidden);
72     relu_inplace(hidden);
73     linear_i8(model.output, hidden, logits);
74
75     InferenceResult result;
76     result.predicted_class = argmax(logits);
77     result.logits = std::move(logits);
78     return result;
79 }
80
81 } // namespace lcqi

```

Listing 16.13 src/int8_kernels.cpp

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #include <lcqi/inference.hpp>
4
5 #include <algorithm>
6 #include <chrono>
7 #include <cmath>
8 #include <stdexcept>
9 #include <string>
10
11 namespace lcqi {
12     namespace {
13
14         constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
15         constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
16         constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
17         constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;

```

```

18 constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
19 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
20 constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
21 constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
22 constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
23 constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
24 constexpr const char* LCQI_BACKEND_AVX2 = "avx2";
25
26 void require_positive(std::int32_t value, const char* name) {
27     if (value <= 0) {
28         throw std::runtime_error(std::string(name) + " must be positive");
29     }
30 }
31
32 std::size_t checked_size(std::int32_t value, const char* name) {
33     if (value < 0) {
34         throw std::runtime_error(std::string(name) + " cannot be negative");
35     }
36     return static_cast<std::size_t>(value);
37 }
38
39 void validate_quantized_layer(const QuantizedLinearLayer& layer) {
40     require_positive(layer.input_size, "input_size");
41     require_positive(layer.output_size, "output_size");
42     const std::size_t expected_weights =
43         checked_size(layer.input_size, "input_size") *
44         checked_size(layer.output_size, "output_size");
45     if (layer.weights.size() != expected_weights) {
46         throw std::runtime_error("quantized layer weight size mismatch");
47     }
48     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("quantized layer bias size mismatch");
50     }
51 }
52
53 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
54     require_positive(value, "value");
55     require_positive(block_size, "block_size");
56     return ((value + block_size - 1) / block_size) * block_size;
57 }
58
59 float checksum(std::span<const float> values) {
60     float sum = 0.0F;
61     for (const float value : values) {
62         sum += value;
63     }
64     return sum;
65 }
66
67 void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
68     result.backends.push_back(KernelBackendBenchmark{
69         .name = name,

```

```

70     .available = false,
71     .average_us = 0.0,
72     .max_abs_diff = 0.0F,
73     .checksum = 0.0F,
74 });
75 }
76
77 template <typename Callable>
78 double measure_average_us(std::int32_t repeat, Callable&& callable) {
79     require_positive(repeat, "repeat");
80     const auto begin = std::chrono::steady_clock::now();
81     for (std::int32_t index = 0; index < repeat; ++index) {
82         callable();
83     }
84     const auto end = std::chrono::steady_clock::now();
85     const std::chrono::duration<double> elapsed = end - begin;
86     return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>←
    ↪ >(repeat);
87 }
88
89 } // namespace
90
91 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
92                               std::int32_t output_block_size) {
93     validate_quantized_layer(layer);
94     require_positive(output_block_size, "output_block_size");
95
96     PackedLinearI8 packed;
97     packed.input_size = layer.input_size;
98     packed.output_size = layer.output_size;
99     packed.output_block_size = output_block_size;
100    packed.scale = layer.scale;
101    packed.bias = layer.bias;
102
103    const std::int32_t padded_outputs =
104        round_up_to_block(layer.output_size, output_block_size);
105    packed.packed_weights.assign(
106        checked_size(padded_outputs, "padded_outputs") *
107        checked_size(layer.input_size, "input_size"),
108        0);
109
110    std::size_t packed_index = 0;
111    for (std::int32_t output_block = 0; output_block < padded_outputs;
112         output_block += output_block_size) {
113        for (std::int32_t input_index = 0; input_index < layer.input_size; ++←
    ↪ input_index) {
114            for (std::int32_t offset = 0; offset < output_block_size; ++offset)←
    ↪ {
115                const std::int32_t output_index = output_block + offset;
116                if (output_index < layer.output_size) {
117                    const std::size_t source =
118                        checked_size(output_index, "output_index") *

```



```

119         checked_size(layer.input_size, "input_size") +
120         checked_size(input_index, "input_index");
121         packed.packed_weights[packed_index] = layer.weights[source↵
↵ ];
122     }
123     ++packed_index;
124 }
125 }
126 }
127 return packed;
128 }
129
130 void linear_i8_packed(const PackedLinearI8& layer,
131                      std::span<const float> input,
132                      std::span<float> output) {
133     std::vector<float> accumulators(
134         checked_size(layer.output_block_size, "output_block_size"),
135         0.0F);
136     linear_i8_packed_with_workspace(layer, input, output, accumulators);
137 }
138
139 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
140                                     std::span<const float> input,
141                                     std::span<float> output,
142                                     std::span<float> accumulator_workspace) {
143     require_positive(layer.input_size, "packed input_size");
144     require_positive(layer.output_size, "packed output_size");
145     require_positive(layer.output_block_size, "packed output_block_size");
146     if (input.size() != checked_size(layer.input_size, "input_size")) {
147         throw std::runtime_error("linear_i8_packed input size mismatch");
148     }
149     if (output.size() != checked_size(layer.output_size, "output_size")) {
150         throw std::runtime_error("linear_i8_packed output size mismatch");
151     }
152     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
153         throw std::runtime_error("linear_i8_packed bias size mismatch");
154     }
155
156     const std::int32_t padded_outputs =
157         round_up_to_block(layer.output_size, layer.output_block_size);
158     const std::size_t expected_packed_size =
159         checked_size(padded_outputs, "padded_outputs") *
160         checked_size(layer.input_size, "input_size");
161     if (layer.packed_weights.size() != expected_packed_size) {
162         throw std::runtime_error("linear_i8_packed packed weight size mismatch"↵
↵ );
163     }
164     if (accumulator_workspace.size() != checked_size(layer.output_block_size, "↵
↵ output_block_size")) {
165         throw std::runtime_error("linear_i8_packed accumulator workspace size ↵
↵ mismatch");
166     }

```

```

167
168     std::size_t packed_index = 0;
169     for (std::int32_t output_block = 0; output_block < padded_outputs;
170         output_block += layer.output_block_size) {
171         std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), ←
172         ↪ 0.0F);
173         for (std::int32_t offset = 0; offset < layer.output_block_size; ++↪
174         ↪ offset) {
175             const std::int32_t output_index = output_block + offset;
176             if (output_index < layer.output_size) {
177                 accumulator_workspace[checked_size(offset, "offset")] =
178                 layer.bias[checked_size(output_index, "output_index")];
179             }
180         }
181         for (std::int32_t input_index = 0; input_index < layer.input_size; ++↪
182         ↪ input_index) {
183             const float input_value = input[checked_size(input_index, "↪
184             ↪ input_index")];
185             for (std::int32_t offset = 0; offset < layer.output_block_size; ++↪
186             ↪ offset) {
187                 const std::int32_t output_index = output_block + offset;
188                 const std::int8_t weight = layer.packed_weights[packed_index↪
189                 ↪ ++];
190                 if (output_index < layer.output_size) {
191                     accumulator_workspace[checked_size(offset, "offset")] +=
192                     static_cast<float>(weight) * layer.scale * input_value;
193                 }
194             }
195         }
196         for (std::int32_t offset = 0; offset < layer.output_block_size; ++↪
197         ↪ offset) {
198             const std::int32_t output_index = output_block + offset;
199             if (output_index < layer.output_size) {
200                 output[checked_size(output_index, "output_index")] =
201                 accumulator_workspace[checked_size(offset, "offset")];
202             }
203         }
204     }
205 }
206
207 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
208                                                  std::int32_t output_size,
209                                                  float scale) {
210     require_positive(input_size, "input_size");
211     require_positive(output_size, "output_size");
212     QuantizedLinearLayer layer;
213     layer.input_size = input_size;
214     layer.output_size = output_size;
215     layer.scale = scale;
216     const std::size_t weight_count =

```

```

212     checked_size(input_size, "input_size") * checked_size(output_size, "↵
↵ output_size");
213     layer.weights.resize(weight_count);
214     layer.bias.resize(checked_size(output_size, "output_size"));
215     for (std::int32_t output_index = 0; output_index < output_size; ++↵
↵ output_index) {
216         layer.bias[checked_size(output_index, "output_index")] =
217             static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
218                               LCQI_INPUT_PATTERN_CENTER) *
219             LCQI_INPUT_PATTERN_SCALE;
220         for (std::int32_t input_index = 0; input_index < input_size; ++↵
↵ input_index) {
221             const std::int32_t raw =
222                 ((output_index * input_size + input_index) % ↵
↵ LCQI_I8_PATTERN_MODULUS) -
223                 LCQI_I8_PATTERN_CENTER;
224             layer.weights[checked_size(output_index, "output_index") *
225                           checked_size(input_size, "input_size") +
226                           checked_size(input_index, "input_index")] =
227                 static_cast<std::int8_t>(raw);
228         }
229     }
230     return layer;
231 }
232
233 std::vector<float> make_deterministic_input(std::int32_t input_size) {
234     require_positive(input_size, "input_size");
235     std::vector<float> input(checked_size(input_size, "input_size"));
236     for (std::int32_t input_index = 0; input_index < input_size; ++input_index)↵
↵ {
237         input[checked_size(input_index, "input_index")] =
238             static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
239                               LCQI_INPUT_PATTERN_CENTER) *
240             LCQI_INPUT_PATTERN_SCALE;
241     }
242     return input;
243 }
244
245 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
246     if (lhs.size() != rhs.size()) {
247         throw std::runtime_error("max_abs_diff size mismatch");
248     }
249     float diff = 0.0F;
250     for (std::size_t index = 0; index < lhs.size(); ++index) {
251         diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
252     }
253     return diff;
254 }
255
256 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↵
↵ benchmark_case) {
257     require_positive(benchmark_case.input_size, "benchmark input_size");

```

```

258     require_positive(benchmark_case.output_size, "benchmark output_size");
259     require_positive(benchmark_case.output_block_size, "benchmark
↪ output_block_size");
260     require_positive(benchmark_case.repeat, "benchmark repeat");
261
262     const QuantizedLinearLayer layer =
263         make_deterministic_i8_layer(benchmark_case.input_size,
264                                     benchmark_case.output_size,
265                                     0.015625F);
266     const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.↪
↪ output_block_size);
267     const PackedLinearI8 packed_simd = pack_linear_i8(layer, ↪
↪ LCQI_SIMD_OUTPUT_BLOCK);
268     const std::vector<float> input = make_deterministic_input(benchmark_case.↪
↪ input_size);
269     std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "↪
↪ output_size"),
270                                     0.0F);
271     std::vector<float> packed_output(scalar_output.size(), 0.0F);
272     std::vector<float> simd_output(scalar_output.size(), 0.0F);
273     std::vector<float> packed_workspace(
274         checked_size(benchmark_case.output_block_size, "output_block_size"),
275         0.0F);
276
277     linear_i8(layer, input, scalar_output);
278     linear_i8_packed_with_workspace(packed, input, packed_output, ↪
↪ packed_workspace);
279
280     KernelBenchmarkResult result;
281     result.benchmark_case = benchmark_case;
282     result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283     result.backends.push_back(KernelBackendBenchmark{
284         .name = LCQI_BACKEND_SCALAR,
285         .available = true,
286         .average_us = measure_average_us(benchmark_case.repeat,
287                                         [&]() { linear_i8(layer, input, ↪
↪ scalar_output); }),
288         .max_abs_diff = 0.0F,
289         .checksum = checksum(scalar_output),
290     });
291     result.backends.push_back(KernelBackendBenchmark{
292         .name = LCQI_BACKEND_PACKED_SCALAR,
293         .available = true,
294         .average_us = measure_average_us(
295             benchmark_case.repeat,
296             [&]() { linear_i8_packed_with_workspace(packed,
297                                                         input,
298                                                         packed_output,
299                                                         packed_workspace); }),
300         .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301         .checksum = checksum(packed_output),
302     });

```

```

303     if (linear_i8_packed_avx2_available()) {
304         linear_i8_packed_avx2(packed_simd, input, simd_output);
305         result.backends.push_back(KernelBackendBenchmark{
306             .name = LCQI_BACKEND_AVX2,
307             .available = true,
308             .average_us = measure_average_us(
309                 benchmark_case.repeat,
310                 [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); }
311             ),
312             .max_abs_diff = max_abs_diff(scalar_output, simd_output),
313             .checksum = checksum(simd_output),
314         });
315     } else {
316         add_unavailable_backend(result, LCQI_BACKEND_AVX2);
317     }
318     return result;
319 }
320 } // namespace lcqi
321
322 #if !defined(LCQI_ENABLE_AVX2)
323 namespace lcqi {
324
325 bool linear_i8_packed_avx2_available() noexcept {
326     return false;
327 }
328
329 void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
330     throw std::runtime_error("AVX2 packed kernel is not enabled in this build");
331 }
332
333 } // namespace lcqi
334 #endif

```

Listing 16.14 src/int8_kernels_avx2.cpp

```

1  #include <lcqi/int8_kernels.hpp>
2
3  #if defined(LCQI_ENABLE_AVX2)
4
5  #include <immintrin.h>
6
7  #include <algorithm>
8  #include <cstdint>
9  #include <cstring>
10 #include <stdexcept>
11 #include <string>
12
13 namespace lcqi {
14 namespace {

```

```

15
16 constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;
17
18 void require_positive(std::int32_t value, const char* name) {
19     if (value <= 0) {
20         throw std::runtime_error(std::string(name) + " must be positive");
21     }
22 }
23
24 std::size_t checked_size(std::int32_t value, const char* name) {
25     if (value < 0) {
26         throw std::runtime_error(std::string(name) + " cannot be negative");
27     }
28     return static_cast<std::size_t>(value);
29 }
30
31 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
32     require_positive(value, "value");
33     require_positive(block_size, "block_size");
34     return ((value + block_size - 1) / block_size) * block_size;
35 }
36
37 void validate_avx2_contract(const PackedLinearI8& layer,
38                             std::span<const float> input,
39                             std::span<float> output) {
40     require_positive(layer.input_size, "packed input_size");
41     require_positive(layer.output_size, "packed output_size");
42     if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
43         throw std::runtime_error("AVX2 packed kernel requires output_block_size ←
44     ↪ == 8");
45     }
46     if (input.size() != checked_size(layer.input_size, "input_size")) {
47         throw std::runtime_error("AVX2 packed kernel input size mismatch");
48     }
49     if (output.size() != checked_size(layer.output_size, "output_size")) {
50         throw std::runtime_error("AVX2 packed kernel output size mismatch");
51     }
52     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
53         throw std::runtime_error("AVX2 packed kernel bias size mismatch");
54     }
55     const std::int32_t padded_outputs =
56         round_up_to_block(layer.output_size, layer.output_block_size);
57     const std::size_t expected_packed_size =
58         checked_size(padded_outputs, "padded_outputs") *
59         checked_size(layer.input_size, "input_size");
60     if (layer.packed_weights.size() != expected_packed_size) {
61         throw std::runtime_error("AVX2 packed kernel packed weight size ←
62     ↪ mismatch");
63     }
64 } // namespace

```

```

65
66 bool linear_i8_packed_avx2_available() noexcept {
67     #if defined(__GNUC__) || defined(__clang__)
68         __builtin_cpu_init();
69         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
70     #else
71         return true;
72     #endif
73 }
74
75 void linear_i8_packed_avx2(const PackedLinearI8& layer,
76                             std::span<const float> input,
77                             std::span<float> output) {
78     if (!linear_i8_packed_avx2_available()) {
79         throw std::runtime_error("AVX2/FMA is not available on this CPU");
80     }
81     validate_avx2_contract(layer, input, output);
82
83     const std::int32_t padded_outputs =
84         round_up_to_block(layer.output_size, layer.output_block_size);
85     std::size_t packed_index = 0;
86     alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};
87
88     for (std::int32_t output_block = 0; output_block < padded_outputs;
89         output_block += LCQI_AVX2_OUTPUT_BLOCK) {
90         __m256 accumulator = _mm256_setzero_ps();
91
92         for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
93             const std::int32_t output_index = output_block + offset;
94             block_output[checked_size(offset, "offset")] =
95                 output_index < layer.output_size
96                 ? layer.bias[checked_size(output_index, "output_index")]
97                 : 0.0F;
98         }
99         accumulator = _mm256_load_ps(block_output);
100
101         for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
102             std::uint64_t packed_bytes = 0;
103             std::memcpy(&packed_bytes,
104                         layer.packed_weights.data() + packed_index,
105                         sizeof(packed_bytes));
106             const __m128i packed_i8 =
107                 _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
108             packed_index += LCQI_AVX2_OUTPUT_BLOCK;
109             const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
110             const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
111             const __m256 input_scaled =
112                 _mm256_set1_ps(input[checked_size(input_index, "input_index")])
113                 * layer.scale;
114             accumulator = _mm256_fmadd_ps(weights_f32, input_scaled,

```



```

    ↪ accumulator);
114     }
115
116     _mm256_store_ps(block_output, accumulator);
117     for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) ↪
    ↪ ) {
118         const std::int32_t output_index = output_block + offset;
119         if (output_index < layer.output_size) {
120             output[checked_size(output_index, "output_index")] =
121                 block_output[checked_size(offset, "offset")];
122         }
123     }
124 }
125 }
126
127 } // namespace lcqi
128
129 #endif

```

Listing 16.15 src/model.cpp

```

1 #include <lcqi/model.hpp>
2
3 #include <fstream>
4 #include <limits>
5 #include <stdexcept>
6 #include <string>
7
8 namespace lcqi {
9     namespace {
10
11         constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
12         constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;
13
14         std::string read_key(std::istream& input) {
15             std::string key;
16             if (!(input >> key)) {
17                 throw std::runtime_error("unexpected end of model file");
18             }
19             return key;
20         }
21
22         void expect_key(std::istream& input, const std::string& expected) {
23             const std::string key = read_key(input);
24             if (key != expected) {
25                 throw std::runtime_error("expected key '" + expected + "', got '" + key ↪
                ↪ + "'");
26             }
27         }
28
29         std::int32_t read_i32(std::istream& input, const std::string& key) {
30             expect_key(input, key);

```

```

31     std::int32_t value = 0;
32     if (!(input >> value)) {
33         throw std::runtime_error("invalid int32 value for key '" + key + "'");
34     }
35     return value;
36 }
37
38 float read_float(std::istream& input, const std::string& key) {
39     expect_key(input, key);
40     float value = 0.0F;
41     if (!(input >> value)) {
42         throw std::runtime_error("invalid float value for key '" + key + "'");
43     }
44     return value;
45 }
46
47 std::vector<std::int8_t> read_i8_vector(std::istream& input,
48                                         const std::string& key,
49                                         std::int32_t count) {
50     expect_key(input, key);
51     if (count < 0) {
52         throw std::runtime_error("negative vector size for key '" + key + "'");
53     }
54     std::vector<std::int8_t> values(static_cast<std::size_t>(count));
55     for (std::int32_t i = 0; i < count; ++i) {
56         std::int32_t raw = 0;
57         if (!(input >> raw)) {
58             throw std::runtime_error("invalid int8 payload for key '" + key + "←
59             ↪ '");
60         }
61         if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
62             throw std::runtime_error("int8 payload out of range for key '" + ←
63             ↪ key + "'");
64         }
65         values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
66     }
67     return values;
68 }
69
70 std::vector<float> read_float_vector(std::istream& input,
71                                       const std::string& key,
72                                       std::int32_t count) {
73     expect_key(input, key);
74     if (count < 0) {
75         throw std::runtime_error("negative vector size for key '" + key + "'");
76     }
77     std::vector<float> values(static_cast<std::size_t>(count));
78     for (std::int32_t i = 0; i < count; ++i) {
79         if (!(input >> values[static_cast<std::size_t>(i)])) {
80             throw std::runtime_error("invalid float payload for key '" + key + "←
81             ↪ '");
82         }
83     }
84 }

```

```

80     }
81     return values;
82 }
83
84 void validate_layer(const QuantizedLinearLayer& layer, const std::string& name)↵
    ↵ {
85     if (layer.input_size <= 0 || layer.output_size <= 0) {
86         throw std::runtime_error(name + " layer has invalid shape");
87     }
88     const std::size_t expected_weights =
89         static_cast<std::size_t>(layer.input_size) *
90         static_cast<std::size_t>(layer.output_size);
91     if (layer.weights.size() != expected_weights) {
92         throw std::runtime_error(name + " layer weight size mismatch");
93     }
94     if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
95         throw std::runtime_error(name + " layer bias size mismatch");
96     }
97 }
98
99 std::int32_t checked_element_count(std::int32_t rows,
100                                   std::int32_t columns,
101                                   const std::string& name) {
102     if (rows <= 0 || columns <= 0) {
103         throw std::runtime_error(name + " layer has invalid shape");
104     }
105     const std::int64_t count =
106         static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
107     if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::↵
    ↵ max())) {
108         throw std::runtime_error(name + " layer element count is too large");
109     }
110     return static_cast<std::int32_t>(count);
111 }
112
113 } // namespace
114
115 TinyMlpModel load_model(const std::filesystem::path& path) {
116     std::ifstream input(path);
117     if (!input) {
118         throw std::runtime_error("cannot open model file: " + path.string());
119     }
120
121     expect_key(input, "LCQI_MODEL_V1");
122
123     TinyMlpModel model;
124     model.input_size = read_i32(input, "input_size");
125     model.hidden_size = read_i32(input, "hidden_size");
126     model.output_size = read_i32(input, "output_size");
127
128     const std::int32_t hidden_weight_count =
129         checked_element_count(model.input_size, model.hidden_size, "hidden");

```

```

130     const std::int32_t output_weight_count =
131         checked_element_count(model.hidden_size, model.output_size, "output");
132
133     model.hidden.input_size = model.input_size;
134     model.hidden.output_size = model.hidden_size;
135     model.hidden.scale = read_float(input, "hidden_scale");
136     model.hidden.weights = read_i8_vector(
137         input,
138         "hidden_weights",
139         hidden_weight_count);
140     model.hidden.bias = read_float_vector(input, "hidden_bias", model.↵
↵ hidden_size);
141
142     model.output.input_size = model.hidden_size;
143     model.output.output_size = model.output_size;
144     model.output.scale = read_float(input, "output_scale");
145     model.output.weights = read_i8_vector(
146         input,
147         "output_weights",
148         output_weight_count);
149     model.output.bias = read_float_vector(input, "output_bias", model.↵
↵ output_size);
150
151     validate_layer(model.hidden, "hidden");
152     validate_layer(model.output, "output");
153     return model;
154 }
155
156 } // namespace lcqi

```

Listing 16.16 src/reference_decoder.cpp

```

1  #include <lcqi/reference_decoder.hpp>
2
3  #include <algorithm>
4  #include <array>
5  #include <cmath>
6  #include <limits>
7  #include <stdexcept>
8  #include <string>
9
10 namespace lcqi {
11 namespace {
12
13 constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::↵
↵ infinity();
14 constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
15 constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
16 constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
17 constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
18 constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";
19

```

```

20 void require_positive(std::int32_t value, const char* name) {
21     if (value <= 0) {
22         throw std::runtime_error(std::string(name) + " must be positive");
23     }
24 }
25
26 void validate_config(const DecoderConfig& config) {
27     require_positive(config.hidden_size, "hidden_size");
28     require_positive(config.query_heads, "query_heads");
29     require_positive(config.kv_heads, "kv_heads");
30     require_positive(config.head_dim, "head_dim");
31     require_positive(config.intermediate_size, "intermediate_size");
32     require_positive(config.vocab_size, "vocab_size");
33     require_positive(config.max_context, "max_context");
34     if (config.query_heads % config.kv_heads != 0) {
35         throw std::runtime_error("query_heads must be divisible by kv_heads");
36     }
37     if (config.hidden_size != config.query_heads * config.head_dim) {
38         throw std::runtime_error("hidden_size must equal query_heads * head_dim");
39     }
40     if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
41         throw std::runtime_error("head_dim must be even for RoPE");
42     }
43     if (config.rope_base <= 1.0F) {
44         throw std::runtime_error("rope_base must be greater than 1");
45     }
46 }
47
48 std::size_t checked_size(std::int32_t value, const char* name) {
49     if (value < 0) {
50         throw std::runtime_error(std::string(name) + " cannot be negative");
51     }
52     return static_cast<std::size_t>(value);
53 }
54
55 std::int32_t kv_width(const DecoderConfig& config) {
56     return config.kv_heads * config.head_dim;
57 }
58
59 void validate_linear(const LinearWeightsF32& layer, const char* name) {
60     require_positive(layer.input_size, "linear input_size");
61     require_positive(layer.output_size, "linear output_size");
62     const std::size_t expected_weights =
63         checked_size(layer.input_size, name) * checked_size(layer.output_size,
64         ↪ name);
65     if (layer.weights.size() != expected_weights) {
66         throw std::runtime_error(std::string(name) + " weight size mismatch");
67     }
68     if (!layer.bias.empty() &&
69         layer.bias.size() != checked_size(layer.output_size, name)) {
70         throw std::runtime_error(std::string(name) + " bias size mismatch");
71     }
72 }

```

```

70     }
71 }
72
73 void validate_span_size(std::size_t actual, std::int32_t expected, const char* ↵
    ↵ name) {
74     if (actual != checked_size(expected, name)) {
75         throw std::runtime_error(std::string(name) + " size mismatch");
76     }
77 }
78
79 std::int32_t argmax(std::span<const float> values) {
80     if (values.empty()) {
81         throw std::runtime_error("cannot take argmax of empty values");
82     }
83     std::int32_t best_index = 0;
84     float best_value = values[0];
85     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()↵
    ↵ ); ++index) {
86         const float candidate = values[static_cast<std::size_t>(index)];
87         if (candidate > best_value) {
88             best_value = candidate;
89             best_index = index;
90         }
91     }
92     return best_index;
93 }
94
95 float silu(float value) {
96     return value / (1.0F + std::exp(-value));
97 }
98
99 std::span<const float> row_span(std::span<const float> values,
100                                std::int32_t row,
101                                std::int32_t row_width) {
102     const std::size_t begin = checked_size(row, "row") * checked_size(row_width, ↵
    ↵ , "row_width");
103     return values.subspan(begin, checked_size(row_width, "row_width"));
104 }
105
106 void add_inplace(std::span<float> target, std::span<const float> source) {
107     if (target.size() != source.size()) {
108         throw std::runtime_error("residual add size mismatch");
109     }
110     for (std::size_t index = 0; index < target.size(); ++index) {
111         target[index] += source[index];
112     }
113 }
114
115 void swiglu_mlp(const DecoderLayerWeightsF32& layer,
116                std::span<const float> input,
117                std::span<float> output) {
118     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0↵

```

```

↪ F);
119     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
120     std::vector<float> hidden(gate.size(), 0.0F);
121     linear_f32(layer.w_gate, input, gate);
122     linear_f32(layer.w_up, input, up);
123     if (gate.size() != up.size()) {
124         throw std::runtime_error("SwiGLU gate/up size mismatch");
125     }
126     for (std::size_t index = 0; index < hidden.size(); ++index) {
127         hidden[index] = silu(gate[index]) * up[index];
128     }
129     linear_f32(layer.w_down, hidden, output);
130 }
131
132 float checksum(std::span<const float> values) {
133     float sum = 0.0F;
134     for (const float value : values) {
135         sum += value;
136     }
137     return sum;
138 }
139
140 float max_abs(std::span<const float> values) {
141     float result = 0.0F;
142     for (const float value : values) {
143         result = std::max(result, std::fabs(value));
144     }
145     return result;
146 }
147
148 std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape↪
↪ ) {
149     std::vector<std::int32_t> stride(shape.size(), 1);
150     std::int32_t running = 1;
151     for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; ↪
↪ index >= 0; --index) {
152         stride[checked_size(index, "stride index")] = running;
153         running *= shape[checked_size(index, "shape index")];
154     }
155     return stride;
156 }
157
158 void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
159                    const char* name,
160                    std::int32_t token_position,
161                    std::int32_t layer_id,
162                    std::span<const std::int32_t> shape,
163                    const char* layout,
164                    std::span<const float> values) {
165     if (checkpoints == nullptr) {
166         return;
167     }

```



```

168     ReferenceTensorCheckpoint checkpoint;
169     checkpoint.name = name;
170     checkpoint.token_position = token_position;
171     checkpoint.layer_id = layer_id;
172     checkpoint.shape.assign(shape.begin(), shape.end());
173     checkpoint.stride = contiguous_stride(shape);
174     checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
175     checkpoint.layout = layout;
176     checkpoint.checksum = checksum(values);
177     checkpoint.max_abs = max_abs(values);
178     checkpoint.values.assign(values.begin(), values.end());
179     checkpoints->push_back(std::move(checkpoint));
180 }
181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183                           std::span<const float> input,
184                           std::span<float> output,
185                           std::int32_t token_position,
186                           std::int32_t layer_id,
187                           std::vector<ReferenceTensorCheckpoint>* checkpoints)↵
188 ↵ {
189     std::vector<float> gate(check_size(layer.w_gate.output_size, "gate"), 0.0↵
190 ↵ F);
191     std::vector<float> up(check_size(layer.w_up.output_size, "up"), 0.0F);
192     std::vector<float> hidden(gate.size(), 0.0F);
193     linear_f32(layer.w_gate, input, gate);
194     linear_f32(layer.w_up, input, up);
195     if (gate.size() != up.size()) {
196         throw std::runtime_error("SwiGLU gate/up size mismatch");
197     }
198     const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↵
199 ↵ output_size};
200     add_checkpoint(checkpoints,
201                   "mlp_gate",
202                   token_position,
203                   layer_id,
204                   intermediate_shape,
205                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
206                   gate);
207     add_checkpoint(checkpoints,
208                   "mlp_up",
209                   token_position,
210                   layer_id,
211                   intermediate_shape,
212                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
213                   up);
214     for (std::size_t index = 0; index < hidden.size(); ++index) {
215         hidden[index] = silu(gate[index]) * up[index];
216     }
217     add_checkpoint(checkpoints,
218                   "mlp_hidden",
219                   token_position,

```

```

217         layer_id,
218         intermediate_shape,
219         LCQI_TRACE_LAYOUT_CONTIGUOUS,
220         hidden);
221     linear_f32(layer.w_down, hidden, output);
222 }
223
224 void forward_layer(const DecoderConfig& config,
225                  const DecoderLayerWeightsF32& layer,
226                  std::int32_t layer_id,
227                  std::int32_t model_position,
228                  ReferenceKVCache& cache,
229                  std::span<float> hidden,
230                  std::vector<ReferenceTensorCheckpoint>* checkpoints = ←
    ↪ nullptr) {
231     validate_span_size(hidden.size(), config.hidden_size, "hidden");
232
233     std::vector<float> normed(checkered_size(config.hidden_size, "hidden_size"), ←
    ↪ 0.0F);
234     std::vector<float> query(checkered_size(config.hidden_size, "hidden_size"), ←
    ↪ 0.0F);
235     std::vector<float> key(checkered_size(kv_width(config), "kv_width"), 0.0F);
236     std::vector<float> value(checkered_size(kv_width(config), "kv_width"), 0.0F);
237     std::vector<float> attention(checkered_size(config.hidden_size, "hidden_size" ←
    ↪ ), 0.0F);
238     std::vector<float> attention_projected(checkered_size(config.hidden_size, " ←
    ↪ hidden_size"), 0.0F);
239     std::vector<float> mlp(checkered_size(config.hidden_size, "hidden_size"), 0.0 ←
    ↪ F);
240
241     const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242     const std::array<std::int32_t, 2> query_shape{config.query_heads, config. ←
    ↪ head_dim};
243     const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim ←
    ↪ };
244     const std::array<std::int32_t, 4> kv_slot_shape{
245         1, 1, config.kv_heads, config.head_dim};
246
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249                   "attn_norm",
250                   model_position,
251                   layer_id,
252                   hidden_shape,
253                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
254                   normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259                   "q_before_rope",
260                   model_position,

```

```

261         layer_id,
262         query_shape,
263         LCQI_TRACE_LAYOUT_CONTIGUOUS,
264         query);
265     add_checkpoint(checkpoints,
266                   "k_before_rope",
267                   model_position,
268                   layer_id,
269                   kv_shape,
270                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
271                   key);
272     add_checkpoint(checkpoints,
273                   "v",
274                   model_position,
275                   layer_id,
276                   kv_shape,
277                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
278                   value);
279     apply_rope(query, config.query_heads, config.head_dim, model_position, ←
    ↪ config.rope_base);
280     apply_rope(key, config.kv_heads, config.head_dim, model_position, config.←
    ↪ rope_base);
281     add_checkpoint(checkpoints,
282                   "q_after_rope",
283                   model_position,
284                   layer_id,
285                   query_shape,
286                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
287                   query);
288     add_checkpoint(checkpoints,
289                   "k_after_rope",
290                   model_position,
291                   layer_id,
292                   kv_shape,
293                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
294                   key);
295
296     cache.append(layer_id, model_position, key, value);
297     add_checkpoint(checkpoints,
298                   "kv_cache_key_slot",
299                   model_position,
300                   layer_id,
301                   kv_slot_shape,
302                   LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
303                   key);
304     attention_decode(config, cache, layer_id, model_position, query, attention)←
    ↪ ;
305     add_checkpoint(checkpoints,
306                   "attention_out",
307                   model_position,
308                   layer_id,
309                   hidden_shape,

```

```

310         LCQI_TRACE_LAYOUT_CONTIGUOUS,
311         attention);
312 linear_f32(layer.wo, attention, attention_projected);
313 add_inplace(hidden, attention_projected);
314 add_checkpoint(checkpoints,
315                 "post_attention_residual",
316                 model_position,
317                 layer_id,
318                 hidden_shape,
319                 LCQI_TRACE_LAYOUT_CONTIGUOUS,
320                 hidden);
321
322 rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
323 add_checkpoint(checkpoints,
324                 "mlp_norm",
325                 model_position,
326                 layer_id,
327                 hidden_shape,
328                 LCQI_TRACE_LAYOUT_CONTIGUOUS,
329                 normed);
330 if (checkpoints == nullptr) {
331     swiglu_mlp(layer, normed, mlp);
332 } else {
333     swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ↵
↵ checkpoints);
334 }
335 add_inplace(hidden, mlp);
336 add_checkpoint(checkpoints,
337                 "layer_out",
338                 model_position,
339                 layer_id,
340                 hidden_shape,
341                 LCQI_TRACE_LAYOUT_CONTIGUOUS,
342                 hidden);
343 }
344
345 LinearWeightsF32 make_linear(std::int32_t input_size,
346                             std::int32_t output_size,
347                             std::vector<float> weights,
348                             std::vector<float> bias = {}) {
349     LinearWeightsF32 layer;
350     layer.input_size = input_size;
351     layer.output_size = output_size;
352     layer.weights = std::move(weights);
353     layer.bias = std::move(bias);
354     validate_linear(layer, "tiny linear");
355     return layer;
356 }
357
358 std::vector<float> zeros(std::int32_t count) {
359     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
360 }

```

```

361
362 } // namespace
363
364 ReferenceDecodeTraceResult run_reference_decode_with_trace(
365     const ReferenceDecoderModel& model,
366     std::span<const std::int32_t> token_ids) {
367     validate_config(model.config);
368     if (token_ids.empty()) {
369         throw std::runtime_error("reference decode needs at least one token");
370     }
371     if (token_ids.size() > checked_size(model.config.max_context, "max_context"
↵ )) {
372         throw std::runtime_error("token_ids exceed max_context");
373     }
374     if (model.token_embedding.size() !=
375         checked_size(model.config.vocab_size, "vocab_size") *
376         checked_size(model.config.hidden_size, "hidden_size")) {
377         throw std::runtime_error("token embedding size mismatch");
378     }
379     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size
↵ , "hidden_size")) {
380         throw std::runtime_error("final norm weight size mismatch");
381     }
382     validate_linear(model.lm_head, "lm_head");
383
384     ReferenceDecodeTraceResult trace;
385     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers
↵ .size()));
386     std::vector<float> hidden(checked_size(model.config.hidden_size, "
↵ hidden_size"), 0.0F);
387     const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
388     for (std::int32_t position = 0; position < static_cast<std::int32_t>(
↵ token_ids.size());
389         ++position) {
390         const std::int32_t token_id = token_ids[checked_size(position, "
↵ position")];
391         if (token_id < 0 || token_id >= model.config.vocab_size) {
392             throw std::runtime_error("token id out of vocabulary");
393         }
394         const std::span<const float> embedding =
395             row_span(model.token_embedding, token_id, model.config.hidden_size)
↵ ;
396         std::copy(embedding.begin(), embedding.end(), hidden.begin());
397         add_checkpoint(&trace.checkpoints,
398             "embedding",
399             position,
400             -1,
401             hidden_shape,
402             LCQI_TRACE_LAYOUT_ROW_MAJOR,
403             hidden);
404         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(
↵ model.layers.size());

```

```

405         ++layer_id) {
406             forward_layer(model.config,
407                           model.layers[checked_size(layer_id, "layer_id")],
408                           layer_id,
409                           position,
410                           cache,
411                           hidden,
412                           &trace.checkpoints);
413     }
414 }
415
416     std::vector<float> normed(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
417     std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size↵
↵ "), 0.0F);
418     const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
419     const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size↵
↵ ()) - 1;
420     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↵
↵ ;
421     add_checkpoint(&trace.checkpoints,
422                   "final_norm",
423                   last_position,
424                   -1,
425                   hidden_shape,
426                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
427                   normed);
428     linear_f32(model.lm_head, normed, logits);
429     add_checkpoint(&trace.checkpoints,
430                   "logits",
431                   last_position,
432                   -1,
433                   logits_shape,
434                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
435                   logits);
436
437     trace.result.predicted_token = argmax(logits);
438     trace.result.logits = std::move(logits);
439     return trace;
440 }
441
442 ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t ↵
↵ layer_count)
443 : config_(config),
444   layer_count_(layer_count) {
445     validate_config(this->config_);
446     require_positive(this->layer_count_, "layer_count");
447     const std::size_t element_count =
448         checked_size(this->layer_count_, "layer_count") *
449         checked_size(this->config_.max_context, "max_context") *
450         checked_size(this->config_.kv_heads, "kv_heads") *
451         checked_size(this->config_.head_dim, "head_dim");

```

[illegible]


```

497     this->validate_address(layer_id, model_position, kv_head);
498     const std::size_t offset = this->base_offset(layer_id, model_position, ←
    ↪ kv_head);
499     return std::span<const float>(this->values_.data() + offset,
500                                   checked_size(this->config_.head_dim, "←
    ↪ head_dim"));
501 }
502
503 std::int32_t ReferenceKVCache::layer_count() const noexcept {
504     return this->layer_count_;
505 }
506
507 std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
508     return this->filled_tokens_;
509 }
510
511 std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
512                                           std::int32_t model_position,
513                                           std::int32_t kv_head) const {
514     return (((checked_size(layer_id, "layer_id") *
515                checked_size(this->config_.max_context, "max_context")) +
516             checked_size(model_position, "model_position")) *
517            checked_size(this->config_.kv_heads, "kv_heads") +
518            checked_size(kv_head, "kv_head")) *
519            checked_size(this->config_.head_dim, "head_dim");
520 }
521
522 void ReferenceKVCache::validate_address(std::int32_t layer_id,
523                                         std::int32_t model_position,
524                                         std::int32_t kv_head) const {
525     if (layer_id < 0 || layer_id >= this->layer_count_) {
526         throw std::runtime_error("KV layer_id out of range");
527     }
528     if (model_position < 0 || model_position >= this->config_.max_context) {
529         throw std::runtime_error("KV model_position out of range");
530     }
531     if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
532         throw std::runtime_error("KV head out of range");
533     }
534 }
535
536 void rms_norm(std::span<const float> input,
537              std::span<const float> weight,
538              float epsilon,
539              std::span<float> output) {
540     if (input.size() != weight.size() || input.size() != output.size()) {
541         throw std::runtime_error("RMSNorm size mismatch");
542     }
543     if (input.empty()) {
544         throw std::runtime_error("RMSNorm input is empty");
545     }
546     float sum_square = 0.0F;

```

```

547     for (const float value : input) {
548         sum_square += value * value;
549     }
550     const float mean_square = sum_square / static_cast<float>(input.size());
551     const float scale = 1.0F / std::sqrt(mean_square + epsilon);
552     for (std::size_t index = 0; index < input.size(); ++index) {
553         output[index] = input[index] * scale * weight[index];
554     }
555 }
556
557 void linear_f32(const LinearWeightsF32& layer,
558                std::span<const float> input,
559                std::span<float> output) {
560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↵
↵ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↵
↵ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                 input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576                std::int32_t head_count,
577                std::int32_t head_dim,
578                std::int32_t model_position,
579                float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += ↵
↵ LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<↵
↵ float>(head_dim);

```

```

595         const float theta = 1.0F / std::pow(rope_base, exponent);
596         const float angle = static_cast<float>(model_position) * theta;
597         const float cosine = std::cos(angle);
598         const float sine = std::sin(angle);
599         const std::size_t first =
600             checked_size(head, "head") * checked_size(head_dim, "head_dim")
        ↪ +
601             checked_size(offset, "offset");
602         const float x0 = heads[first];
603         const float x1 = heads[first + 1];
604         heads[first] = x0 * cosine - x1 * sine;
605         heads[first + 1] = x0 * sine + x1 * cosine;
606     }
607 }
608 }
609
610 void attention_decode(const DecoderConfig& config,
611                     const ReferenceKVCache& cache,
612                     std::int32_t layer_id,
613                     std::int32_t model_position,
614                     std::span<const float> query,
615                     std::span<float> output) {
616     validate_config(config);
617     validate_span_size(query.size(), config.hidden_size, "attention query");
618     validate_span_size(output.size(), config.hidden_size, "attention output");
619     if (model_position < 0 || model_position >= cache.filled_tokens()) {
620         throw std::runtime_error("attention model_position has not been written
        ↪ ");
621     }
622     std::fill(output.begin(), output.end(), 0.0F);
623
624     const std::int32_t group_size = config.query_heads / config.kv_heads;
625     const float score_scale = 1.0F / std::sqrt(static_cast<float>(config
        ↪ head_dim));
626     std::vector<float> scores(checked_size(model_position + 1, "score count"),
627                             LCQI_SOFTMAX_NEGATIVE_INFINITY);
628     std::vector<float> probabilities(scores.size(), 0.0F);
629
630     for (std::int32_t query_head = 0; query_head < config.query_heads; ++
        ↪ query_head) {
631         const std::int32_t kv_head = query_head / group_size;
632         const std::size_t query_base =
633             checked_size(query_head, "query_head") * checked_size(config
        ↪ head_dim, "head_dim");
634         float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
635         for (std::int32_t position = 0; position <= model_position; ++position)
        ↪ {
636             const std::span<const float> key = cache.key(layer_id, position,
        ↪ kv_head);
637             float dot = 0.0F;
638             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
639                 dot += query[query_base + checked_size(dim, "dim")] *

```

```

640         key[checked_size(dim, "dim")];
641     }
642     const float score = dot * score_scale;
643     scores[checked_size(position, "position")] = score;
644     max_score = std::max(max_score, score);
645 }
646
647 float probability_sum = 0.0F;
648 for (std::int32_t position = 0; position <= model_position; ++position) ←
↪ {
649     const std::size_t index = checked_size(position, "position");
650     const float unnormalized = std::exp(scores[index] - max_score);
651     probabilities[index] = unnormalized;
652     probability_sum += unnormalized;
653 }
654 if (probability_sum <= 0.0F) {
655     throw std::runtime_error("attention probability sum is invalid");
656 }
657 for (std::int32_t position = 0; position <= model_position; ++position) ←
↪ {
658     const float probability =
659         probabilities[checked_size(position, "position")] / ←
↪ probability_sum;
660     const std::span<const float> value = cache.value(layer_id, position ←
↪ , kv_head);
661     for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662         output[query_base + checked_size(dim, "dim")] +=
663             probability * value[checked_size(dim, "dim")];
664     }
665 }
666 }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670     std::span<const std::int32_t> ←
↪ token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context" ←
↪ )) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680         checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size ←
↪ , "hidden_size")) {
684         throw std::runtime_error("final norm weight size mismatch");

```

```

685     }
686     validate_linear(model.lm_head, "lm_head");
687
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers-<
    ↪ .size()));
689     std::vector<float> hidden(check_size(model.config.hidden_size, "↪
    ↪ hidden_size"), 0.0F);
690     for (std::int32_t position = 0; position < static_cast<std::int32_t>(↪
    ↪ token_ids.size());
691         ++position) {
692         const std::int32_t token_id = token_ids[checked_size(position, "↪
    ↪ position")];
693         if (token_id < 0 || token_id >= model.config.vocab_size) {
694             throw std::runtime_error("token id out of vocabulary");
695         }
696         const std::span<const float> embedding =
697             row_span(model.token_embedding, token_id, model.config.hidden_size)↪
    ↪ ;
698         std::copy(embedding.begin(), embedding.end(), hidden.begin());
699         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↪
    ↪ model.layers.size());
700             ++layer_id) {
701             forward_layer(model.config,
702                           model.layers[checked_size(layer_id, "layer_id")],
703                           layer_id,
704                           position,
705                           cache,
706                           hidden);
707         }
708     }
709
710     std::vector<float> normed(check_size(model.config.hidden_size, "↪
    ↪ hidden_size"), 0.0F);
711     std::vector<float> logits(check_size(model.config.vocab_size, "vocab_size↪
    ↪ "), 0.0F);
712     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↪
    ↪ ;
713     linear_f32(model.lm_head, normed, logits);
714
715     ReferenceDecodeResult result;
716     result.predicted_token = argmax(logits);
717     result.logits = std::move(logits);
718     return result;
719 }
720
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;

```

```

728 model.config.vocab_size = 5;
729 model.config.max_context = 8;
730 model.config.rms_epsilon = 1.0e-5F;
731 model.config.rope_base = 10000.0F;
732
733 model.token_embedding = {
734     0.20F, -0.10F, 0.05F, 0.30F,
735     -0.40F, 0.10F, 0.25F, -0.20F,
736     0.15F, 0.35F, -0.30F, 0.05F,
737     0.50F, -0.25F, 0.10F, -0.15F,
738     -0.05F, 0.20F, 0.40F, -0.35F,
739 };
740
741 DecoderLayerWeightsF32 layer;
742 layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
743 layer.wq = make_linear(4,
744     4,
745     {
746         0.20F, -0.10F, 0.05F, 0.30F,
747         -0.15F, 0.25F, 0.10F, -0.05F,
748         0.05F, 0.10F, -0.20F, 0.15F,
749         0.30F, -0.05F, 0.20F, 0.10F,
750     });
751 layer.wk = make_linear(4,
752     2,
753     {
754         0.10F, 0.20F, -0.10F, 0.05F,
755         -0.05F, 0.15F, 0.25F, -0.20F,
756     });
757 layer.wv = make_linear(4,
758     2,
759     {
760         0.25F, -0.05F, 0.10F, 0.15F,
761         -0.10F, 0.30F, 0.05F, 0.20F,
762     });
763 layer.wo = make_linear(4,
764     4,
765     {
766         0.20F, 0.10F, -0.05F, 0.15F,
767         -0.10F, 0.25F, 0.20F, -0.05F,
768         0.05F, -0.20F, 0.30F, 0.10F,
769         0.15F, 0.05F, -0.10F, 0.25F,
770     });
771 layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
772 layer.w_gate = make_linear(4,
773     6,
774     {
775         0.20F, -0.10F, 0.05F, 0.10F,
776         -0.05F, 0.15F, 0.20F, -0.10F,
777         0.10F, 0.05F, -0.15F, 0.25F,
778         -0.20F, 0.10F, 0.15F, 0.05F,
779         0.05F, -0.25F, 0.10F, 0.20F,

```

```

780         0.15F, 0.05F, 0.05F, -0.15F,
781     });
782     layer.w_up = make_linear(4,
783         6,
784         {
785             0.10F, 0.20F, -0.05F, 0.05F,
786             -0.15F, 0.05F, 0.25F, 0.10F,
787             0.20F, -0.10F, 0.05F, 0.15F,
788             0.05F, 0.10F, -0.20F, 0.25F,
789             -0.05F, 0.15F, 0.10F, -0.10F,
790             0.25F, -0.05F, 0.05F, 0.10F,
791         });
792     layer.w_down = make_linear(6,
793         4,
794         {
795             0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
796             -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
797             0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
798             0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
799         });
800     model.layers.push_back(std::move(layer));
801     model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
802     model.lm_head = make_linear(4,
803         5,
804         {
805             0.20F, -0.10F, 0.05F, 0.15F,
806             -0.05F, 0.25F, 0.10F, -0.20F,
807             0.15F, 0.05F, -0.25F, 0.10F,
808             -0.10F, 0.20F, 0.15F, 0.05F,
809             0.05F, -0.15F, 0.20F, 0.25F,
810         },
811         zeros(5));
812     return model;
813 }
814
815 } // namespace lcqi

```

Listing 16.17 src/safetensors.cpp

```

1  #include <lcqi/safetensors.hpp>
2
3  #include <algorithm>
4  #include <cctype>
5  #include <cstdint>
6  #include <fstream>
7  #include <limits>
8  #include <stdexcept>
9  #include <string>
10 #include <string_view>
11
12 namespace lcqi {
13 namespace {

```



```

14
15 constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
16 constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U ←
    ↪ ;
17 constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
18 constexpr std::uint64_t LCQI_BYTE_BITS = 8;
19 constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
20 constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
21 constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
22 constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
23 constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
24 constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
25 constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
26 constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
27 constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
28 constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
29 constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
30
31 class HeaderCursor {
32 public:
33     explicit HeaderCursor(std::string_view text) : text_(text) {}
34
35     void expect(char expected) {
36         this->skip_ws();
37         if (this->eof() || this->text_[this->position_] != expected) {
38             throw std::runtime_error("invalid safetensors header syntax");
39         }
40         ++this->position_;
41     }
42
43     [[nodiscard]] bool consume(char expected) {
44         this->skip_ws();
45         if (!this->eof() && this->text_[this->position_] == expected) {
46             ++this->position_;
47             return true;
48         }
49         return false;
50     }
51
52     [[nodiscard]] std::string parse_string() {
53         this->skip_ws();
54         this->expect('"');
55         std::string result;
56         while (!this->eof()) {
57             const char ch = this->text_[this->position_++];
58             if (ch == '"') {
59                 return result;
60             }
61             if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) ←
        ↪ {
62                 throw std::runtime_error("control character in safetensors ←
        ↪ string");

```

```

63         }
64         if (ch == '\\') {
65             result.push_back(this->parse_escape());
66         } else {
67             result.push_back(ch);
68         }
69     }
70     throw std::runtime_error("unterminated safetensors string");
71 }
72
73 [[nodiscard]] std::uint64_t parse_u64() {
74     this->skip_ws();
75     if (this->eof() ||
76         !std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
77         throw std::runtime_error("expected unsigned integer in safetensors ↵
↵ header");
78     }
79     std::uint64_t value = 0;
80     while (!this->eof() &&
81         std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
82         const std::uint64_t digit =
83             static_cast<std::uint64_t>(this->text_[this->position_] - '0');
84         if (value >
85             (std::numeric_limits<std::uint64_t>::max() - digit) / ↵
↵ LCQI_DECIMAL_BASE) {
86             throw std::runtime_error("safetensors integer overflow");
87         }
88         value = value * LCQI_DECIMAL_BASE + digit;
89         ++this->position_;
90     }
91     return value;
92 }
93
94 [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
95     std::vector<std::int64_t> values;
96     this->expect('[');
97     if (this->consume(' ')) {
98         return values;
99     }
100     while (true) {
101         const std::uint64_t raw = this->parse_u64();
102         if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::↵
↵ int64_t>::max())) {
103             throw std::runtime_error("safetensors shape value is too large"↵
↵ );
104         }
105         values.push_back(static_cast<std::int64_t>(raw));
106         if (this->consume(' ')) {
107             return values;
108         }

```

```

109         this->expect(',');
110     }
111 }
112
113 [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
114     std::vector<std::uint64_t> values;
115     this->expect('[');
116     if (this->consume(' ')) {
117         return values;
118     }
119     while (true) {
120         values.push_back(this->parse_u64());
121         if (this->consume(' ')) {
122             return values;
123         }
124         this->expect(',');
125     }
126 }
127
128 [[nodiscard]] bool eof_after_ws() {
129     this->skip_ws();
130     return this->eof();
131 }
132
133 private:
134     std::string_view text_;
135     std::size_t position_ = 0;
136
137     [[nodiscard]] bool eof() const {
138         return this->position_ >= this->text_.size();
139     }
140
141     void skip_ws() {
142         while (!this->eof() &&
143             std::isspace(static_cast<unsigned char>(this->text_[this->position_])) {
144             ++this->position_;
145         }
146     }
147
148     [[nodiscard]] char parse_escape() {
149         if (this->eof()) {
150             throw std::runtime_error("unterminated safetensors escape");
151         }
152         const char escaped = this->text_[this->position_++];
153         switch (escaped) {
154             case '"':
155             case '\\':
156             case '/':
157                 return escaped;
158             case 'b':
159                 return '\b';

```

```

160         case 'f':
161             return '\f';
162         case 'n':
163             return '\n';
164         case 'r':
165             return '\r';
166         case 't':
167             return '\t';
168         default:
169             throw std::runtime_error("unsupported safetensors string escape↵
↵ ");
170     }
171 }
172 };
173
174 std::uint64_t read_le_u64(const std::string& bytes) {
175     if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
176         throw std::runtime_error("safetensors file is too small");
177     }
178     std::uint64_t value = 0;
179     for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
180         value |= static_cast<std::uint64_t>(<
static_cast<unsigned char>(bytes[i]))
181             << (LCQI_BYTE_BITS * i);
182     }
183     return value;
184 }
185 }
186
187 std::string read_file_bytes(const std::filesystem::path& path) {
188     std::ifstream input(path, std::ios::binary);
189     if (!input) {
190         throw std::runtime_error("cannot open safetensors file: " + path.string↵
↵ ());
191     }
192     input.seekg(0, std::ios::end);
193     const std::streamoff size = input.tellg();
194     if (size < 0) {
195         throw std::runtime_error("cannot determine safetensors file size");
196     }
197     input.seekg(0, std::ios::beg);
198     std::string bytes(static_cast<std::size_t>(size), '\0');
199     if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize>↵
↵ >(bytes.size())) {
200         throw std::runtime_error("cannot read safetensors file");
201     }
202     return bytes;
203 }
204
205 void parse_metadata_value(HeaderCursor& cursor,
206                           SafeTensorManifest& manifest,
207                           const std::string& key) {
208     if (key == "__metadata__") {

```

```

209     cursor.expect('{');
210     if (cursor.consume('}')) {
211         return;
212     }
213     while (true) {
214         const std::string metadata_key = cursor.parse_string();
215         cursor.expect(':');
216         const std::string metadata_value = cursor.parse_string();
217         manifest.metadata.emplace_back(metadata_key, metadata_value);
218         if (cursor.consume('}')) {
219             return;
220         }
221         cursor.expect(',');
222     }
223 }
224 throw std::runtime_error("unexpected safetensors metadata value");
225 }
226
227 SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& ↵
↵ name) {
228     SafeTensorEntry entry;
229     entry.name = name;
230     bool saw_dtype = false;
231     bool saw_shape = false;
232     bool saw_offsets = false;
233
234     cursor.expect('{');
235     if (cursor.consume('}')) {
236         throw std::runtime_error("empty safetensors tensor entry");
237     }
238     while (true) {
239         const std::string field = cursor.parse_string();
240         cursor.expect(':');
241         if (field == "dtype") {
242             entry.dtype = cursor.parse_string();
243             saw_dtype = true;
244         } else if (field == "shape") {
245             entry.shape = cursor.parse_i64_array();
246             saw_shape = true;
247         } else if (field == "data_offsets") {
248             const std::vector<std::uint64_t> offsets = cursor.parse_u64_array()↵
↵ ;
249             if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
250                 throw std::runtime_error("safetensors data_offsets must contain↵
↵ two values");
251             }
252             entry.data_begin = offsets[0];
253             entry.data_end = offsets[1];
254             saw_offsets = true;
255         } else {
256             throw std::runtime_error("unsupported safetensors tensor field: " +↵
↵ field);

```

```

257     }
258     if (cursor.consume('}')) {
259         break;
260     }
261     cursor.expect(',');
262 }
263
264 if (!saw_dtype || !saw_shape || !saw_offsets) {
265     throw std::runtime_error("safetensors tensor entry is missing required ↵
↵ fields");
266 }
267 if (entry.data_end < entry.data_begin) {
268     throw std::runtime_error("safetensors tensor offset range is invalid");
269 }
270 return entry;
271 }
272
273 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
274     HeaderCursor cursor(header);
275     cursor.expect('{');
276     if (cursor.consume('}')) {
277         throw std::runtime_error("safetensors header is empty");
278     }
279
280     while (true) {
281         const std::string key = cursor.parse_string();
282         cursor.expect(':');
283         if (key == "__metadata__") {
284             parse_metadata_value(cursor, manifest, key);
285         } else {
286             manifest.tensors.push_back(parse_tensor_entry(cursor, key));
287         }
288         if (cursor.consume('}')) {
289             break;
290         }
291         cursor.expect(',');
292     }
293
294     if (!cursor.eof_after_ws()) {
295         throw std::runtime_error("trailing data after safetensors header");
296     }
297 }
298
299 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
300
301 void validate_manifest(const SafeTensorManifest& manifest,
302                       std::uint64_t file_size) {
303     const std::uint64_t payload_size = file_size - manifest.data_start_offset;
304     for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
305         const SafeTensorEntry& left = manifest.tensors[i];
306         if (left.name.empty() || left.dtype.empty()) {
307             throw std::runtime_error("safetensors tensor has empty name or ↵

```

```

    ↪ dtype");
308     }
309     if (left.data_end > payload_size) {
310         throw std::runtime_error("safetensors tensor data_offsets exceed ↪
    ↪ payload size");
311     }
312     const std::uint64_t expected_bytes = expected_tensor_bytes(left);
313     if (left.byte_size() != expected_bytes) {
314         throw std::runtime_error("safetensors tensor byte size does not ↪
    ↪ match dtype and shape");
315     }
316     for (const std::int64_t dim : left.shape) {
317         if (dim < 0) {
318             throw std::runtime_error("safetensors shape dimension is ↪
    ↪ negative");
319         }
320     }
321     for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
322         const SafeTensorEntry& right = manifest.tensors[j];
323         if (left.name == right.name) {
324             throw std::runtime_error("duplicate safetensors tensor name");
325         }
326         const bool overlaps =
327             left.data_begin < right.data_end && right.data_begin < left.↪
    ↪ data_end;
328         if (overlaps) {
329             throw std::runtime_error("overlapping safetensors tensor data ↪
    ↪ range");
330         }
331     }
332 }
333 }
334
335 std::uint64_t dtype_byte_size(std::string_view dtype) {
336     if (dtype == "F64") {
337         return LCQI_DTYPE_BYTES_F64;
338     }
339     if (dtype == "F32") {
340         return LCQI_DTYPE_BYTES_F32;
341     }
342     if (dtype == "F16" || dtype == "BF16") {
343         return LCQI_DTYPE_BYTES_F16;
344     }
345     if (dtype == "I64" || dtype == "U64") {
346         return LCQI_DTYPE_BYTES_I64;
347     }
348     if (dtype == "I32" || dtype == "U32") {
349         return LCQI_DTYPE_BYTES_I32;
350     }
351     if (dtype == "I16" || dtype == "U16") {
352         return LCQI_DTYPE_BYTES_I16;
353     }

```

```

354     if (dtype == "I8") {
355         return LCQI_DTYPE_BYTES_I8;
356     }
357     if (dtype == "U8" || dtype == "BOOL") {
358         return LCQI_DTYPE_BYTES_U8;
359     }
360     throw std::runtime_error("unsupported safetensors dtype: " + std::string(dtype));
361 }
362
363 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
364     std::uint64_t element_count = 1;
365     for (const std::int64_t dim : entry.shape) {
366         if (dim < 0) {
367             throw std::runtime_error("safetensors shape dimension is negative");
368         }
369         const std::uint64_t extent = static_cast<std::uint64_t>(dim);
370         if (extent != 0 &&
371             element_count > std::numeric_limits<std::uint64_t>::max() / extent)
372             throw std::runtime_error("safetensors shape element count overflow");
373         element_count *= extent;
374     }
375     const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
376     if (dtype_bytes != 0 &&
377         element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes)
378         throw std::runtime_error("safetensors tensor byte size overflow");
379     return element_count * dtype_bytes;
380 }
381 } // namespace
382
383 std::uint64_t SafeTensorEntry::byte_size() const {
384     return this->data_end - this->data_begin;
385 }
386
387 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) {
388     const auto found = std::find_if(
389         this->tensors.begin(),
390         this->tensors.end(),
391         [name](const SafeTensorEntry& entry) {
392             return entry.name == name;
393         });
394     if (found == this->tensors.end()) {
395         return nullptr;
396     }
397 }

```



```

400     return &(*found);
401 }
402
403 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path)↵
404     ↵ {
405     const std::string bytes = read_file_bytes(path);
406     const std::uint64_t header_size = read_le_u64(bytes);
407     if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
408         throw std::runtime_error("safetensors header size is invalid");
409     }
410     const std::uint64_t data_start_offset =
411         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size↵
412     ↵ ;
413     if (data_start_offset > bytes.size()) {
414         throw std::runtime_error("safetensors header extends beyond file size")↵
415     ↵ ;
416     }
417
418     SafeTensorManifest manifest;
419     manifest.header_size = header_size;
420     manifest.data_start_offset = data_start_offset;
421     const std::string_view header(
422         bytes.data() + LCQI_SAFETENSORS_PREFIX_BYTES,
423         static_cast<std::size_t>(header_size));
424     parse_header(header, manifest);
425     validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
426     return manifest;
427 }
428
429 } // namespace lcqi

```

Listing 16.18 src/tokenizer.cpp

```

1  #include <lcqi/tokenizer.hpp>
2
3  #include <cstdint>
4  #include <fstream>
5  #include <stdexcept>
6  #include <string>
7  #include <string_view>
8
9  namespace lcqi {
10 namespace {
11
12 constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 1469598103934665603ULL;
13 constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
14 constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;
15
16 std::string read_key(std::istream& input) {
17     std::string key;
18     if (!(input >> key)) {
19         throw std::runtime_error("unexpected end of tokenizer file");

```

```

20     }
21     return key;
22 }
23
24 void expect_key(std::istream& input, const std::string& expected) {
25     const std::string key = read_key(input);
26     if (key != expected) {
27         throw std::runtime_error("expected tokenizer key '" + expected + "', ←
↪ got '" + key + "'");
28     }
29 }
30
31 std::int32_t read_i32(std::istream& input, const std::string& key) {
32     expect_key(input, key);
33     std::int32_t value = 0;
34     if (!(input >> value)) {
35         throw std::runtime_error("invalid tokenizer int32 value for key '" + ←
↪ key + "'");
36     }
37     return value;
38 }
39
40 std::string read_string(std::istream& input, const std::string& key) {
41     expect_key(input, key);
42     std::string value;
43     if (!(input >> value)) {
44         throw std::runtime_error("invalid tokenizer string value for key '" + ←
↪ key + "'");
45     }
46     return value;
47 }
48
49 std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view ←
↪ text) {
50     for (const TokenEntry& entry : tokenizer.vocab) {
51         if (entry.text == text) {
52             return entry.id;
53         }
54     }
55     return LCQI_INVALID_TOKEN_ID;
56 }
57
58 void validate_tokenizer(const TokenizerModel& tokenizer) {
59     if (tokenizer.vocab.empty()) {
60         throw std::runtime_error("tokenizer vocab is empty");
61     }
62     for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
63         const TokenEntry& left = tokenizer.vocab[i];
64         if (left.id < 0) {
65             throw std::runtime_error("tokenizer token id must be non-negative")←
↪ ;
66     }

```

```

67     for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
68         const TokenEntry& right = tokenizer.vocab[j];
69         if (left.id == right.id) {
70             throw std::runtime_error("duplicate tokenizer token id");
71         }
72         if (left.text == right.text) {
73             throw std::runtime_error("duplicate tokenizer token text");
74         }
75     }
76 }
77 if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
78     lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
79     throw std::runtime_error("tokenizer BOS id does not match vocab");
80 }
81 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
82     lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
83     throw std::runtime_error("tokenizer EOS id does not match vocab");
84 }
85 }
86
87 std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
88     hash ^= static_cast<std::uint64_t>(value);
89     hash *= LCQI_FNV_PRIME;
90     return hash;
91 }
92
93 std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
94     for (const unsigned char value : text) {
95         hash = hash_byte(hash, value);
96     }
97     return hash;
98 }
99
100 std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
101     const std::uint32_t bits = static_cast<std::uint32_t>(value);
102     for (std::int32_t shift = 0; shift < 32; shift += 8) {
103         hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0x
104         ↪ FFU));
105     }
106     return hash;
107 }
108 } // namespace
109
110 TokenizerModel load_tokenizer(const std::filesystem::path& path) {
111     std::ifstream input(path);
112     if (!input) {
113         throw std::runtime_error("cannot open tokenizer file: " + path.string()
114         ↪ );
115     }
116     expect_key(input, "LCQI_TOKENIZER_V1");

```

```

117
118     TokenizerModel tokenizer;
119     tokenizer.bos_id = read_i32(input, "bos_id");
120     tokenizer.eos_id = read_i32(input, "eos_id");
121     tokenizer.unk_id = read_i32(input, "unk_id");
122     tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
123     const std::int32_t vocab_size = read_i32(input, "vocab_size");
124     if (vocab_size <= 0) {
125         throw std::runtime_error("tokenizer vocab_size must be positive");
126     }
127
128     tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
129     expect_key(input, "vocab");
130     for (std::int32_t i = 0; i < vocab_size; ++i) {
131         TokenEntry entry;
132         if (!(input >> entry.id >> entry.text)) {
133             throw std::runtime_error("invalid tokenizer vocab entry");
134         }
135         tokenizer.vocab.push_back(entry);
136     }
137
138     validate_tokenizer(tokenizer);
139     return tokenizer;
140 }
141
142 std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
143                                       std::string_view prompt) {
144     std::vector<std::int32_t> ids;
145     if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
146         ids.push_back(tokenizer.bos_id);
147     }
148
149     std::size_t begin = 0;
150     while (begin < prompt.size()) {
151         while (begin < prompt.size() && prompt[begin] == ' ') {
152             ++begin;
153         }
154         if (begin >= prompt.size()) {
155             break;
156         }
157         std::size_t end = begin;
158         while (end < prompt.size() && prompt[end] != ' ') {
159             ++end;
160         }
161         const std::string_view token = prompt.substr(begin, end - begin);
162         const std::int32_t token_id = lookup_token_id(tokenizer, token);
163         if (token_id == LCQI_INVALID_TOKEN_ID) {
164             if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
165                 throw std::runtime_error("tokenizer encountered unknown token")↵
166             ↵ ;
167             }
168             ids.push_back(tokenizer.unk_id);

```

```

168         } else {
169             ids.push_back(token_id);
170         }
171         begin = end;
172     }
173
174     if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
175         ids.push_back(tokenizer.eos_id);
176     }
177     return ids;
178 }
179
180 std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
181     std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
182     hash = hash_string(hash, tokenizer.chat_template_hash);
183     hash = hash_i32(hash, tokenizer.bos_id);
184     hash = hash_i32(hash, tokenizer.eos_id);
185     hash = hash_i32(hash, tokenizer.unk_id);
186     for (const TokenEntry& entry : tokenizer.vocab) {
187         hash = hash_string(hash, entry.text);
188         hash = hash_i32(hash, entry.id);
189     }
190     return hash;
191 }
192
193 } // namespace lcqi

```

四、命令行、账本、Trace 和 Benchmark

Listing 16.19 src/main.cpp

```

1  #include <lcqi/inference.hpp>
2  #include <lcqi/model.hpp>
3
4  #include <chrono>
5  #include <cstdint>
6  #include <cstdlib>
7  #include <filesystem>
8  #include <iostream>
9  #include <span>
10 #include <stdexcept>
11 #include <string>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
17 constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
18 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
19

```

```

20 std::vector<float> parse_input(int argc, char** argv, std::int32_t ↵
    ↵ expected_size) {
21     if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
22         throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↵
    ↵ [repeat]");
23     }
24     std::vector<float> input(static_cast<std::size_t>(expected_size));
25     for (std::int32_t i = 0; i < expected_size; ++i) {
26         input[static_cast<std::size_t>(i)] =
27             std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
28     }
29     return input;
30 }
31
32 std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
33     const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
34     if (argc <= repeat_index) {
35         return LCQI_DEFAULT_REPEAT;
36     }
37     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
    ↵ repeat_index]));
38     if (repeat <= 0) {
39         throw std::runtime_error("repeat must be positive");
40     }
41     return repeat;
42 }
43
44 } // namespace
45
46 int main(int argc, char** argv) {
47     try {
48         if (argc < LCQI_EXPECTED_ARGC_MIN) {
49             throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats↵
    ↵ ...> [repeat]");
50         }
51
52         const std::filesystem::path model_path = argv[1];
53         const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
54         const std::vector<float> input = parse_input(argc, argv, model.↵
    ↵ input_size);
55         const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);
56
57         const lcqi::InferenceResult result = lcqi::run_inference(model, input);
58         std::cout << "predicted_class " << result.predicted_class << "\n";
59         std::cout << "logits";
60         for (const float value : result.logits) {
61             std::cout << ' ' << value;
62         }
63         std::cout << "\n";
64
65         const auto begin = std::chrono::steady_clock::now();
66         std::int32_t checksum = 0;

```

```

67     for (std::int32_t i = 0; i < repeat; ++i) {
68         checksum += lcqi::run_inference(model, input).predicted_class;
69     }
70     const auto end = std::chrono::steady_clock::now();
71     const std::chrono::duration<double> elapsed = end - begin;
72     const double average_us =
73         elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>
74     ↪ >(repeat);
75     std::cout << "repeat " << repeat << "\n";
76     std::cout << "average_us " << average_us << "\n";
77     std::cout << "checksum " << checksum << "\n";
78     return EXIT_SUCCESS;
79 } catch (const std::exception& error) {
80     std::cerr << "error: " << error.what() << "\n";
81     return EXIT_FAILURE;
82 }

```

Listing 16.20 src/bench.cpp

```

1  #include <lcqi/int8_kernels.hpp>
2
3  #include <algorithm>
4  #include <cstdlib>
5  #include <iostream>
6  #include <stdexcept>
7  #include <string>
8  #include <vector>
9
10 namespace {
11
12 constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
13 constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
14 constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
15 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
16 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
17 constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
18 constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
19 constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;
20
21 const char* target_arch() noexcept {
22     #if defined(__x86_64__) || defined(_M_X64)
23         return LCQI_TARGET_ARCH_X86_64;
24     #elif defined(__i386__) || defined(_M_IX86)
25         return LCQI_TARGET_ARCH_I386;
26     #else
27         return LCQI_TARGET_ARCH_UNKNOWN;
28     #endif
29 }
30
31 std::int32_t parse_repeat(int argc, char** argv) {
32     if (argc <= LCQI_REPEAT_ARG_INDEX) {

```

```

33     return LCQI_DEFAULT_REPEAT;
34 }
35 const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
36 if (repeat <= 0) {
37     throw std::runtime_error("repeat must be positive");
38 }
39 return repeat;
40 }
41
42 std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
43     return {
44         {128, 128, 16, repeat},
45         {256, 256, 16, repeat},
46         {512, 512, 16, repeat},
47         {513, 257, 16, repeat},
48         {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
49                                   repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
50         {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
51          std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / ↵
↵ LCQI_LARGE_CASE_REPEAT_DIVISOR)},
52     };
53 }
54
55 } // namespace
56
57 int main(int argc, char** argv) {
58     try {
59         const std::int32_t repeat = parse_repeat(argc, argv);
60         std::cout
61             << "target_arch,input_size,output_size,output_block,repeat,backend,↵
↵ available,"
62             << "average_us,max_abs_diff,checksum\n";
63         for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(↵
↵ repeat)) {
64             const lcqi::KernelBenchmarkResult result =
65                 lcqi::benchmark_linear_i8_case(benchmark_case);
66             for (const lcqi::KernelBackendBenchmark& backend : result.backends)↵
↵ {
67                 std::cout << target_arch() << ','
68                     << result.benchmark_case.input_size << ','
69                     << result.benchmark_case.output_size << ','
70                     << result.benchmark_case.output_block_size << ','
71                     << result.benchmark_case.repeat << ','
72                     << backend.name << ','
73                     << (backend.available ? 1 : 0) << ','
74                     << backend.average_us << ','
75                     << backend.max_abs_diff << ','
76                     << backend.checksum << '\n';
77             }
78         }
79         return EXIT_SUCCESS;

```



```

80     } catch (const std::exception& error) {
81         std::cerr << "error: " << error.what() << '\n';
82         return EXIT_FAILURE;
83     }
84 }

```

Listing 16.21 src/trace.cpp

```

1  #include <lcqi/reference_decoder.hpp>
2  #include <lcqi/tokenizer.hpp>
3
4  #include <algorithm>
5  #include <array>
6  #include <cstdlib>
7  #include <exception>
8  #include <filesystem>
9  #include <iostream>
10 #include <span>
11 #include <sstream>
12 #include <string>
13 #include <string_view>
14
15 namespace {
16
17 constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
18 constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
19 constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
20 constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
21 constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
22 constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;
23
24 std::filesystem::path tokenizer_fixture_path() {
25     return std::filesystem::path(__FILE__).parent_path().parent_path() /
26         "models" / "tiny_tokenizer.txt";
27 }
28
29 std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel& ←
    ↪ & tokenizer,
30                                                     std::string_view prompt) {
31     std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer, ←
    ↪ prompt);
32     if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
33         full_tokens.front() != LCQI_TRACE_BOS_ID ||
34         full_tokens.back() != LCQI_TRACE_EOS_ID) {
35         throw std::runtime_error("unexpected tokenizer fixture ids");
36     }
37
38     std::vector<std::int32_t> decoder_tokens(
39         full_tokens.begin() + 1,
40         full_tokens.end() - 1);
41     if (!std::equal(decoder_tokens.begin(),
42                     decoder_tokens.end(),

```

```

43         LCQI_TRACE_TOKEN_IDS.begin(),
44         LCQI_TRACE_TOKEN_IDS.end())) {
45     throw std::runtime_error("unexpected decoder token ids");
46 }
47 return decoder_tokens;
48 }
49
50 std::string join_ints(std::span<const std::int32_t> values) {
51     std::ostringstream stream;
52     for (std::size_t index = 0; index < values.size(); ++index) {
53         if (index != 0) {
54             stream << 'x';
55         }
56         stream << values[index];
57     }
58     return stream.str();
59 }
60
61 std::string join_int_values(std::span<const std::int32_t> values) {
62     std::ostringstream stream;
63     for (std::size_t index = 0; index < values.size(); ++index) {
64         if (index != 0) {
65             stream << ';';
66         }
67         stream << values[index];
68     }
69     return stream.str();
70 }
71
72 std::int32_t checksum_ints(std::span<const std::int32_t> values) {
73     std::int32_t result = 0;
74     for (const std::int32_t value : values) {
75         result += value;
76     }
77     return result;
78 }
79
80 std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
81     std::int32_t result = 0;
82     for (const std::int32_t value : values) {
83         result = std::max(result, std::abs(value));
84     }
85     return result;
86 }
87
88 std::string join_floats(std::span<const float> values) {
89     std::ostringstream stream;
90     const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
91     for (std::size_t index = 0; index < count; ++index) {
92         if (index != 0) {
93             stream << ';';
94         }

```

```

95     stream << values[index];
96 }
97 if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
98     stream << ";...";
99 }
100 return stream.str();
101 }
102
103 void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
104                    const lcqi::TokenizerModel& tokenizer,
105                    std::span<const std::int32_t> full_tokens) {
106     std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↵
↵ checksum,max_abs,values\n";
107     std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
108                 << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
109     std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
110                 << ', ' << full_tokens.size() << ",1,i32,contiguous,"
111                 << checksum_ints(full_tokens) << ', '
112                 << max_abs_ints(full_tokens) << ', '
113                 << join_int_values(full_tokens) << '\n';
114     std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
115                 << ",scalar,scalar,u64,fnv1a,"
116                 << tokenizer_contract_hash(tokenizer) << ",0,"
117                 << tokenizer.chat_template_hash << '\n';
118     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵     {
119         std::cout << checkpoint.name << ', '
120                 << checkpoint.token_position << ', '
121                 << checkpoint.layer_id << ', '
122                 << join_ints(checkpoint.shape) << ', '
123                 << join_ints(checkpoint.stride) << ', '
124                 << checkpoint.dtype << ', '
125                 << checkpoint.layout << ', '
126                 << checkpoint.checksum << ', '
127                 << checkpoint.max_abs << ', '
128                 << join_floats(checkpoint.values) << '\n';
129     }
130     std::cout << "sampler_argmax,"
131                 << LCQI_TRACE_TOKEN_IDS.size() - 1 << ', '
132                 << LCQI_TRACE_METADATA_LAYER
133                 << ', ' << trace.result.logits.size() << ",1,f32,argmax,"
134                 << "0,0,token=" << trace.result.predicted_token << '\n';
135     std::cout << "generated_token,"
136                 << LCQI_TRACE_TOKEN_IDS.size() << ', '
137                 << LCQI_TRACE_METADATA_LAYER
138                 << ",scalar,scalar,i32,generated,"
139                 << trace.result.predicted_token << ', '
140                 << trace.result.predicted_token << ', '
141                 << trace.result.predicted_token << '\n';
142 }
143
144 } // namespace

```

```

145
146 int main() {
147     try {
148         const lcqi::TokenizerModel tokenizer =
149             lcqi::load_tokenizer(tokenizer_fixture_path());
150         const std::vector<std::int32_t> full_tokens =
151             lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
152         const std::vector<std::int32_t> decoder_tokens =
153             decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
154         const lcqi::ReferenceDecoderModel model = lcqi::↵
↵ make_tiny_reference_decoder_model();
155         const lcqi::ReferenceDecodeTraceResult trace =
156             lcqi::run_reference_decode_with_trace(model, decoder_tokens);
157         print_trace_csv(trace, tokenizer, full_tokens);
158         return EXIT_SUCCESS;
159     } catch (const std::exception& error) {
160         std::cerr << "error: " << error.what() << '\n';
161         return EXIT_FAILURE;
162     }
163 }

```

Listing 16.22 src/ledger.cpp

```

1 #include <cstdlib>
2 #include <cstdint>
3 #include <exception>
4 #include <iomanip>
5 #include <iostream>
6 #include <string_view>
7
8 namespace {
9
10 constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
11 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
12 constexpr double LCQI_FLOPS_PER_MAC = 2.0;
13 constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
14 constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
15 constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
16 constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
17 constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
18 constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
19 constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;
20
21 struct DTypeLedger {
22     std::string_view name;
23     double weight_gb_per_token = 0.0;
24     double kv_element_bytes = 0.0;
25 };
26
27 constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
28     {"fp16", 14.0, 2.0},
29     {"int8", 7.0, 2.0},

```

```

30     {"int4", 3.7, 2.0},
31 };
32
33 constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
34     50.0,
35     100.0,
36     200.0,
37     600.0,
38     1000.0,
39     1600.0,
40 };
41
42 double linear_macs_per_layer() noexcept {
43     const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
44     const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
45     const double qkv = hidden * (hidden + 2.0 * kv_width);
46     const double output_projection = hidden * hidden;
47     const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
48     return qkv + output_projection + mlp;
49 }
50
51 double attention_macs_per_layer() noexcept {
52     return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
53           LCQI_SEVENB_HEAD_DIM;
54 }
55
56 double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
57     return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
58           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
59 }
60
61 double kv_read_gb_per_decode_token(double element_bytes) noexcept {
62     const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
63                           kv_bytes_per_token_all_layers(element_bytes);
64     return bytes / LCQI_BYTES_PER_GB;
65 }
66
67 double kv_capacity_mib(double element_bytes, std::int32_t context_length) ↵
68     ↵ noexcept {
69     const double bytes = static_cast<double>(context_length) *
70                           kv_bytes_per_token_all_layers(element_bytes);
71     return bytes / LCQI_BYTES_PER_MIB;
72 }
73
74 void print_model_rows() {
75     const double linear_macs = linear_macs_per_layer();
76     const double attention_macs = attention_macs_per_layer();
77     const double total_flops =
78         (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * ↵
79         ↵ LCQI_FLOPS_PER_MAC;
80     std::cout << "section,item,value,unit,notes\n";
81     std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ↵

```

```

    ↪ fixture\n";
80     std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
81     std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA,↪
    ↪ query_heads\n";
82     std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV ↪
    ↪ heads\n";
83     std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per ↪
    ↪ head\n";
84     std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs,↪
    ↪ decode token\n";
85     std::cout << "compute,attention_macs_per_layer," << attention_macs
86         << ",macs,context=4096\n";
87     std::cout << "compute,total_decode_flops_per_token," << total_flops
88         << ",flops,linear plus attention\n";
89 }
90
91 void print_kv_rows() {
92     for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93         std::cout << "kv,fp16_capacity_context_" << context << ', '
94             << kv_capacity_mib(2.0, context)
95             << ",MiB,single session\n";
96     }
97 }
98
99 void print_dtype_rows() {
100     for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101         const double kv_read_gb = kv_read_gb_per_decode_token(dtype.↪
    ↪ kv_element_bytes);
102         const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103         std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104             << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105         std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106             << kv_read_gb << ",GB/token,context=4096\n";
107         std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108             << total_gb << ",GB/token,weight plus KV\n";
109         for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110             std::cout << "bandwidth_limit," << dtype.name << "_at_"
111                 << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112                 << bandwidth / total_gb
113                 << ",tokens/s,bandwidth-only upper bound\n";
114         }
115     }
116 }
117
118 } // namespace
119
120 int main() {
121     try {
122         std::cout << std::fixed << std::setprecision(3);
123         print_model_rows();
124         print_kv_rows();
125         print_dtype_rows();

```

```

126         return EXIT_SUCCESS;
127     } catch (const std::exception& error) {
128         std::cerr << "error: " << error.what() << '\n';
129         return EXIT_FAILURE;
130     }
131 }

```

Listing 16.23 src/serving_probe.cpp

```

1  #include <algorithm>
2  #include <cstdlib>
3  #include <cstdint>
4  #include <exception>
5  #include <cmath>
6  #include <iomanip>
7  #include <iostream>
8  #include <string_view>
9  #include <vector>
10
11 namespace {
12
13 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
14 constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
15 constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
16 constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
17 constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
18 constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
19 constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
20 constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
21 constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
22 constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
23 constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
24 constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
25 constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
26 constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;
27
28 struct ProbeRequest {
29     std::string_view request_id;
30     std::int32_t prompt_tokens = 0;
31     std::int32_t max_new_tokens = 0;
32     bool cancel_after_prefill = false;
33 };
34
35 struct ProbeResult {
36     std::string_view request_id;
37     std::int32_t accepted = 0;
38     std::string_view rejection_reason;
39     std::int32_t prompt_tokens = 0;
40     std::int32_t reserved_tokens = 0;
41     double kv_reserved_mib = 0.0;
42     double queue_wait_ms = 0.0;
43     double prefill_ms = 0.0;

```

```

44     double ttft_ms = 0.0;
45     double decode_step_p95_ms = 0.0;
46     double tpot_ms = 0.0;
47     std::int32_t completion_tokens = 0;
48     std::string_view finish_reason;
49     double cancel_reclaim_ms = 0.0;
50 };
51
52 const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
53     {"req_short", 32, 4, false},
54     {"req_rag", 512, 8, false},
55     {"req_cancel", 128, 16, true},
56     {"req_too_long", 4096, 512, false},
57 };
58
59 double kv_mib_for_tokens(std::int32_t tokens) noexcept {
60     const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *
        ↪ 2.0 *
61         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
62         LCQI_PROBE_KV_ELEMENT_BYTES;
63     return bytes / LCQI_BYTES_PER_MIB;
64 }
65
66 double percentile(std::vector<double> values, double fraction) {
67     if (values.empty()) {
68         return 0.0;
69     }
70     std::sort(values.begin(), values.end());
71     const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
72     const std::size_t index =
73         std::min(max_index,
74             static_cast<std::size_t>(
75                 std::ceil(fraction * static_cast<double>(max_index))));
76     return values[index];
77 }
78
79 std::vector<ProbeResult> run_probe() {
80     std::vector<ProbeResult> results;
81     double reserved_kv_mib = 0.0;
82     std::int32_t accepted_before = 0;
83     for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
84         ProbeResult result;
85         result.request_id = request.request_id;
86         result.prompt_tokens = request.prompt_tokens;
87         result.reserved_tokens = request.prompt_tokens + request.max_new_tokens
        ↪ ;
88         result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
89         const double static_session_mib =
90             result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
91             LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
92         if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB
        ↪ ) {

```



```

93         result.accepted = 0;
94         result.rejection_reason = "memory_budget_exceeded";
95         result.finish_reason = "rejected";
96         results.push_back(result);
97         continue;
98     }
99
100     reserved_kv_mib += result.kv_reserved_mib;
101     result.accepted = 1;
102     result.rejection_reason = "none";
103     result.queue_wait_ms = static_cast<double>(accepted_before) *
104                             LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
105     result.prefill_ms = static_cast<double>(request.prompt_tokens) *
106                             LCQI_PROBE_PREFILL_MS_PER_TOKEN;
107     result.ttft_ms = result.queue_wait_ms + result.prefill_ms + ↵
↵ LCQI_PROBE_DECODE_STEP_MS;
108     result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
109                                 static_session_mib / 512.0;
110     result.tpot_ms = result.decode_step_p95_ms;
111     if (request.cancel_after_prefill) {
112         result.completion_tokens = 0;
113         result.finish_reason = "cancelled";
114         result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
115         reserved_kv_mib -= result.kv_reserved_mib;
116     } else {
117         result.completion_tokens = request.max_new_tokens;
118         result.finish_reason = "max_tokens";
119     }
120     ++accepted_before;
121     results.push_back(result);
122 }
123 return results;
124 }
125
126 void print_request_rows(const std::vector<ProbeResult>& results) {
127     std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,↵
↵ reserved_tokens,"
128                 "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,↵
↵ decode_step_p95_ms,"
129                 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,↵
↵ metric_name,"
130                 "metric_value\n";
131     for (const ProbeResult& result : results) {
132         std::cout << "request," << result.request_id << ',' <<
133                     << result.accepted << ',' <<
134                     << result.rejection_reason << ',' <<
135                     << result.prompt_tokens << ',' <<
136                     << result.reserved_tokens << ',' <<
137                     << result.kv_reserved_mib << ',' <<
138                     << result.queue_wait_ms << ',' <<
139                     << result.prefill_ms << ',' <<
140                     << result.ttft_ms << ',' <<

```

```

141         << result.decode_step_p95_ms << ', '
142         << result.tpot_ms << ', '
143         << result.completion_tokens << ', '
144         << result.finish_reason << ', '
145         << result.cancel_reclaim_ms << ",,\n";
146     }
147 }
148
149 void print_summary_row(const std::vector<ProbeResult>& results) {
150     std::vector<double> ttft_values;
151     std::vector<double> queue_values;
152     double accepted = 0.0;
153     double rejected = 0.0;
154     double cancelled = 0.0;
155     double reserved_peak_mib = 0.0;
156     double running_reserved_mib = 0.0;
157     for (const ProbeResult& result : results) {
158         if (result.accepted == 0) {
159             rejected += 1.0;
160             continue;
161         }
162         accepted += 1.0;
163         ttft_values.push_back(result.ttft_ms);
164         queue_values.push_back(result.queue_wait_ms);
165         running_reserved_mib += result.kv_reserved_mib;
166         reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
167         if (result.finish_reason == "cancelled") {
168             cancelled += 1.0;
169             running_reserved_mib -= result.kv_reserved_mib;
170         }
171     }
172     const double total = accepted + rejected;
173     const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
174     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
175         << "accepted_count," << accepted << '\n';
176     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
177         << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled <<
178         << '\n';
179     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
180         << "rejected_count," << rejected << '\n';
181     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
182         << "rejection_rate," << rejection_rate << '\n';
183     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
184         << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
185     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
186         << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n' <<
187         ;
188     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
189         << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
190 } // namespace

```

```

191
192 int main() {
193     try {
194         std::cout << std::fixed << std::setprecision(3);
195         const std::vector<ProbeResult> results = run_probe();
196         print_request_rows(results);
197         print_summary_row(results);
198         return EXIT_SUCCESS;
199     } catch (const std::exception& error) {
200         std::cerr << "error: " << error.what() << '\n';
201         return EXIT_FAILURE;
202     }
203 }

```

Listing 16.24 src/onednn_bench.cpp

```

1 #include <dnnl.hpp>
2
3 #include <chrono>
4 #include <cstdint>
5 #include <cstdlib>
6 #include <iostream>
7 #include <numeric>
8 #include <stdexcept>
9 #include <string>
10 #include <unordered_map>
11 #include <vector>
12
13 namespace {
14
15 constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
16 constexpr std::int64_t LCQI_ONEDNN_K = 4096;
17 constexpr std::int64_t LCQI_ONEDNN_O = 4096;
18 constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
19 constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
20 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
21 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
22 constexpr float LCQI_INPUT_SCALE = 0.125F;
23 constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
24 constexpr std::int32_t LCQI_INPUT_CENTER = 8;
25 constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
26 constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
27 constexpr float LCQI_WEIGHT_SCALE = 0.03125F;
28
29 std::int32_t parse_repeat(int argc, char** argv) {
30     if (argc <= LCQI_REPEAT_ARG_INDEX) {
31         return LCQI_ONEDNN_DEFAULT_REPEAT;
32     }
33     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
34     if (repeat <= 0) {
35         throw std::runtime_error("repeat must be positive");

```

```

36     }
37     return repeat;
38 }
39
40 std::vector<float> make_input() {
41     std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
42     for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
43         input[static_cast<std::size_t>(index)] =
44             static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER) *
45             LCQI_INPUT_SCALE;
46     }
47     return input;
48 }
49
50 std::vector<float> make_weights_kn() {
51     std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K *
52     LCQI_ONEDNN_0));
53     for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
54         for (std::int64_t out = 0; out < LCQI_ONEDNN_0; ++out) {
55             const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
56             const float value =
57                 static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
58                 LCQI_WEIGHT_CENTER) *
59                 LCQI_WEIGHT_SCALE;
60             weights[static_cast<std::size_t>(k * LCQI_ONEDNN_0 + out)] = value;
61         }
62     }
63     return weights;
64 }
65
66 float checksum(const std::vector<float>& values) {
67     return std::accumulate(values.begin(), values.end(), 0.0F);
68 }
69 } // namespace
70
71 int main(int argc, char** argv) {
72     try {
73         const std::int32_t repeat = parse_repeat(argc, argv);
74         std::vector<float> input = make_input();
75         std::vector<float> weights = make_weights_kn();
76         std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH *
77         LCQI_ONEDNN_0),
78             0.0F);
79
80         dnnl::engine engine(dnnl::engine::kind::cpu, 0);
81         dnnl::stream stream(engine);
82
83         const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH,
84         LCQI_ONEDNN_K};
85         const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_0};

```

```

84     const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, ←
↪ LCQI_ONEDNN_0};
85
86     const dnnl::memory::desc source_desc(
87         source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↪
↪ ::ab);
88     const dnnl::memory::desc weight_desc(
89         weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↪
↪ ::ab);
90     const dnnl::memory::desc output_desc(
91         output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↪
↪ ::ab);
92
93     dnnl::memory source_memory(source_desc, engine, input.data());
94     dnnl::memory weight_memory(weight_desc, engine, weights.data());
95     dnnl::memory output_memory(output_desc, engine, output.data());
96
97     dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↪
↪ weight_desc, output_desc);
98     dnnl::matmul matmul(matmul_desc);
99     const std::unordered_map<int, dnnl::memory> arguments = {
100         {DNNL_ARG_SRC, source_memory},
101         {DNNL_ARG_WEIGHTS, weight_memory},
102         {DNNL_ARG_DST, output_memory},
103     };
104
105     for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106         matmul.execute(stream, arguments);
107         stream.wait();
108     }
109
110     const auto begin = std::chrono::steady_clock::now();
111     for (std::int32_t index = 0; index < repeat; ++index) {
112         matmul.execute(stream, arguments);
113         stream.wait();
114     }
115     const auto end = std::chrono::steady_clock::now();
116     const std::chrono::duration<double> elapsed = end - begin;
117     const double average_us =
118         elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>↪
↪ >(repeat);
119
120     std::cout << "library,input_size,output_size,dtype,repeat,average_us,↪
↪ checksum\n";
121     std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_0
122         << ",f32," << repeat << ',' << average_us << ','
123         << checksum(output) << '\n';
124     return EXIT_SUCCESS;
125 } catch (const std::exception& error) {
126     std::cerr << "error: " << error.what() << '\n';
127     return EXIT_FAILURE;
128 }

```

129 }

五、测试和模型 Fixture

Listing 16.25 tests/lcqi_tests.cpp

```

1  #include <lcqi/inference.hpp>
2  #include <lcqi/int8_kernels.hpp>
3  #include <lcqi/model.hpp>
4  #include <lcqi/reference_decoder.hpp>
5  #include <lcqi/safetensors.hpp>
6  #include <lcqi/tokenizer.hpp>
7
8  #include <array>
9  #include <cmath>
10 #include <cstdlib>
11 #include <filesystem>
12 #include <fstream>
13 #include <iostream>
14 #include <span>
15 #include <stdexcept>
16 #include <string>
17 #include <vector>
18
19 namespace {
20
21 constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
22 constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
23 constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
24 constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
25 constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
26 constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
27 constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
28 constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
29 constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
30 constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
31 constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
32 constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
33 constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
34 constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
35 constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
36 constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
37 constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
38 constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
39 constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ←
    ↪ LCQI_REFERENCE_DECODER_LOGITS{
40     0.352516979F,
41     -0.126884326F,
42     0.0797837451F,
43     -0.29797405F,
44     0.127030417F,

```

```

45 };
46 constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
47 constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
48 constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
49 constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 13452902845388333734ULL;
50 constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
51 constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
52 constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
53 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
54 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
55 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
56 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
57 constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
58
59 struct KernelShapeCase {
60     std::int32_t input_size = 0;
61     std::int32_t output_size = 0;
62     std::int32_t output_block_size = 0;
63 };
64
65 constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
66     {17, 19, 8},
67     {33, 7, 8},
68     {64, 32, 16},
69     {65, 31, 16},
70 }};
71
72 void require(bool condition, const char* message) {
73     if (!condition) {
74         throw std::runtime_error(message);
75     }
76 }
77
78 void require_close(float actual, float expected, const char* message) {
79     if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
80         throw std::runtime_error(message);
81     }
82 }
83
84 std::filesystem::path test_model_path() {
85     return std::filesystem::path(__FILE__).parent_path().parent_path() /
86         "models" / "tiny_mlp_i8.txt";
87 }
88
89 std::filesystem::path test_tokenizer_path() {
90     return std::filesystem::path(__FILE__).parent_path().parent_path() /
91         "models" / "tiny_tokenizer.txt";
92 }
93
94 std::filesystem::path temp_file_path(const std::string& name) {
95     return std::filesystem::temp_directory_path() / name;
96 }

```

```

97
98 void write_le_u64(std::ofstream& output, std::uint64_t value) {
99     for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
100         const char byte = static_cast<char>(
101             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
102         output.write(&byte, 1);
103     }
104 }
105
106 void write_safetensors_fixture(const std::filesystem::path& path,
107                               const std::string& header,
108                               std::uint64_t payload_bytes) {
109     std::ofstream output(path, std::ios::binary);
110     if (!output) {
111         throw std::runtime_error("cannot create safetensors test fixture");
112     }
113     write_le_u64(output, static_cast<std::uint64_t>(header.size()));
114     output.write(header.data(), static_cast<std::streamsize>(header.size()));
115     for (std::uint64_t i = 0; i < payload_bytes; ++i) {
116         const char byte = static_cast<char>(i & 0xFFU);
117         output.write(&byte, 1);
118     }
119 }
120
121 void test_tiny_mlp() {
122     const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
123     require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
124     require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch")↵
↵ ;
125     require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch")↵
↵ ;
126
127     const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
128     const lcqi::InferenceResult result = lcqi::run_inference(model, input);
129
130     require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected ↵
↵ predicted class");
131     require(result.logits.size() == static_cast<std::size_t>(↵
↵ LCQI_TEST_OUTPUT_SIZE),
132             "unexpected logits size");
133     require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
134     require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
135
136     std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ↵
↵ 0.0F);
137     lcqi::linear_i8(model.hidden, input, hidden);
138     lcqi::relu_inplace(hidden);
139     require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
140     require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
141     require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
142 }
143

```



```

144 void test_tokenizer_contract() {
145     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↵
↵ test_tokenizer_path());
146     require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
147     require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
148     require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
149     require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
150             "tokenizer template hash mismatch");
151
152     const std::vector<std::int32_t> token_ids =
153         lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
154     const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
155     require(token_ids == expected, "tokenizer prompt ids mismatch");
156     require(lcqi::tokenizer_contract_hash(tokenizer) == ↵
↵ LCQI_TOKENIZER_HASH_EXPECTED,
157             "tokenizer contract hash mismatch");
158 }
159
160 void test_tokenizer_rejects_unknown_without_unk() {
161     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↵
↵ test_tokenizer_path());
162     bool threw = false;
163     try {
164         static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))↵
↵ ;
165     } catch (const std::runtime_error&) {
166         threw = true;
167     }
168     require(threw, "tokenizer should reject unknown token when unk id is absent↵
↵ ");
169 }
170
171 void test_safetensors_manifest_contract() {
172     const std::filesystem::path path =
173         temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
174     const std::string header =
175         R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight"↵
↵ "":{"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.↵
↵ attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets"↵
↵ :[24,48]}})";
176     write_safetensors_fixture(path, header, ↵
↵ LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);
177
178     const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(↵
↵ path);
179     std::filesystem::remove(path);
180
181     require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
182             "safetensors header size mismatch");
183     require(manifest.data_start_offset ==
184             static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
185             static_cast<std::uint64_t>(header.size()),

```

```

186         "safetensors data start mismatch");
187     require(manifest.metadata.size() == 1, "safetensors metadata count mismatch"↵
↵ );
188     require(manifest.metadata[0].first == "format" &&
189             manifest.metadata[0].second == "lcqi-fixture",
190             "safetensors metadata mismatch");
191     require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
192
193     const lcqi::SafeTensorEntry* embedding =
194         manifest.find_tensor("model.embed_tokens.weight");
195     require(embedding != nullptr, "safetensors embedding tensor missing");
196     require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
197     require(embedding->shape == std::vector<std::int64_t>({2, 3}),
198             "safetensors embedding shape mismatch");
199     require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
200             embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
201             "safetensors embedding offset mismatch");
202     require(embedding->byte_size() ==
203             LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
204             "safetensors embedding byte size mismatch");
205
206     const lcqi::SafeTensorEntry* q_proj =
207         manifest.find_tensor("model.layers.0.attn.q_proj.weight");
208     require(q_proj != nullptr, "safetensors q projection tensor missing");
209     require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
210     require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
211             q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
212             "safetensors q projection offset mismatch");
213 }
214
215 void test_safetensors_rejects_bad_offsets() {
216     const std::filesystem::path path =
217         temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
218     const std::string header =
219         R("{"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}}");
220     write_safetensors_fixture(path, header, 4);
221
222     bool threw = false;
223     try {
224         static_cast<void>(lcqi::load_safetensors_manifest(path));
225     } catch (const std::runtime_error&) {
226         threw = true;
227     }
228     std::filesystem::remove(path);
229     require(threw, "safetensors parser should reject offsets beyond payload");
230 }
231
232 void test_safetensors_rejects_bad_dtype_shape_size() {
233     const std::filesystem::path path =
234         temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
235     const std::string header =
236         R("{"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}}");

```

```

237     write_safetensors_fixture(path, header, 8);
238
239     bool threw = false;
240     try {
241         static_cast<void>(lcqi::load_safetensors_manifest(path));
242     } catch (const std::runtime_error&) {
243         threw = true;
244     }
245     std::filesystem::remove(path);
246     require(threw, "safetensors parser should reject dtype/shape byte mismatch" ←
    ↪ );
247 }
248
249 void test_reference_rms_norm() {
250     const std::vector<float> input{3.0F, 4.0F};
251     const std::vector<float> weight{1.0F, 0.5F};
252     std::vector<float> output(2, 0.0F);
253     lcqi::rms_norm(input, weight, 0.0F, output);
254     require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
255     require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
256 }
257
258 void test_reference_rope() {
259     std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
260     lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
261     require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
262     require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
263     require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ←
    ↪ mismatch");
264     require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
265 }
266
267 void test_reference_kv_attention() {
268     lcqi::DecoderConfig config;
269     config.hidden_size = 4;
270     config.query_heads = 2;
271     config.kv_heads = 1;
272     config.head_dim = 2;
273     config.intermediate_size = 4;
274     config.vocab_size = 4;
275     config.max_context = 4;
276
277     lcqi::ReferenceKVCache cache(config, 1);
278     const std::vector<float> key{1.0F, 0.0F};
279     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ←
    ↪ LCQI_ATTENTION_EXPECTED_1};
280     cache.append(0, 0, key, value);
281     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
282     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
283
284     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
285     std::vector<float> output(4, 0.0F);

```

```

286     lcqi::attention_decode(config, cache, 0, 0, query, output);
287     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0 ←
    ↪ mismatch");
288     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1 ←
    ↪ mismatch");
289     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0 ←
    ↪ mismatch");
290     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1 ←
    ↪ mismatch");
291 }
292
293 void test_reference_decoder_end_to_end() {
294     const lcqi::ReferenceDecoderModel model = lcqi:: ←
    ↪ make_tiny_reference_decoder_model();
295     const std::vector<std::int32_t> tokens{1, 2, 3};
296     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model ←
    ↪ , tokens);
297     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
298             "reference decoder logits size mismatch");
299     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
300             "reference decoder predicted token mismatch");
301     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ←
    ↪ ++index) {
302         require_close(result.logits[index],
303                        LCQI_REFERENCE_DECODER_LOGITS[index],
304                        "reference decoder golden logit mismatch");
305     }
306 }
307
308 void test_reference_decoder_trace_contract() {
309     const lcqi::ReferenceDecoderModel model = lcqi:: ←
    ↪ make_tiny_reference_decoder_model();
310     const std::vector<std::int32_t> tokens{1, 2, 3};
311     const lcqi::ReferenceDecodeTraceResult trace =
312         lcqi::run_reference_decode_with_trace(model, tokens);
313
314     require(trace.result.predicted_token == ←
    ↪ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
315             "reference trace predicted token mismatch");
316     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
317
318     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
319     require(first.name == "embedding", "first checkpoint should be embedding");
320     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
321             "first checkpoint token position mismatch");
322     require(first.dtype == "f32", "first checkpoint dtype mismatch");
323     require(first.layout == "row_major", "first checkpoint layout mismatch");
324     require(first.shape.size() == 1 &&
325            first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
326            "first checkpoint shape mismatch");
327     require(first.stride.size() == 1 && first.stride[0] == 1,
328            "first checkpoint stride mismatch");

```

```

329
330     bool saw_logits = false;
331     bool saw_kv_slot = false;
332     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵ {
333         if (checkpoint.name == "kv_cache_key_slot") {
334             saw_kv_slot = true;
335             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch"↵
↵ );
336             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout↵
↵ mismatch");
337         }
338         if (checkpoint.name == "logits") {
339             saw_logits = true;
340             require(checkpoint.token_position == ↵
↵ LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
341                     "logits checkpoint token position mismatch");
342             require(checkpoint.shape.size() == 1 &&
343                     checkpoint.shape[0] ==
344                         static_cast<std::int32_t>(↵
↵ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
345                     "logits checkpoint shape mismatch");
346             require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.↵
↵ size(),
347                     "logits checkpoint value size mismatch");
348             for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.↵
↵ size(); ++index) {
349                 require_close(checkpoint.values[index],
350                             LCQI_REFERENCE_DECODER_LOGITS[index],
351                             "trace logits checkpoint mismatch");
352             }
353         }
354     }
355     require(saw_kv_slot, "KV checkpoint missing");
356     require(saw_logits, "logits checkpoint missing");
357 }
358
359 void test_packed_i8_kernel_matches_scalar() {
360     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
361         const lcqi::QuantizedLinearLayer layer =
362             lcqi::make_deterministic_i8_layer(shape_case.input_size,
363                                             shape_case.output_size,
364                                             0.03125F);
365         const lcqi::PackedLinearI8 packed =
366             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
367         const std::vector<float> input = lcqi::make_deterministic_input(layer.↵
↵ input_size);
368         std::vector<float> scalar_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
369         std::vector<float> packed_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
370         lcqi::linear_i8(layer, input, scalar_output);

```

```

371     lcqi::linear_i8_packed(packed, input, packed_output);
372     require(lcqi::max_abs_diff(scalar_output, packed_output) <= ←
↪ LCQI_KERNEL_MAX_DIFF,
373         "packed int8 kernel output differs from scalar");
374
375     const lcqi::PackedLinearI8 simd_packed =
376         lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
377     if (lcqi::linear_i8_packed_avx2_available()) {
378         std::vector<float> avx2_output(static_cast<std::size_t>(layer.↪
↪ output_size), 0.0F);
379         lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
380         require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ←
↪ LCQI_KERNEL_MAX_DIFF,
381             "AVX2 int8 kernel output differs from scalar");
382     }
383 }
384 }
385
386 } // namespace
387
388 int main() {
389     try {
390         test_tiny_mlp();
391         test_tokenizer_contract();
392         test_tokenizer_rejects_unknown_without_unk();
393         test_safetensors_manifest_contract();
394         test_safetensors_rejects_bad_offsets();
395         test_safetensors_rejects_bad_dtype_shape_size();
396         test_reference_rms_norm();
397         test_reference_rope();
398         test_reference_kv_attention();
399         test_reference_decoder_end_to_end();
400         test_reference_decoder_trace_contract();
401         test_packed_i8_kernel_matches_scalar();
402
403         std::cout << "lcqi tests passed\n";
404         return EXIT_SUCCESS;
405     } catch (const std::exception& error) {
406         std::cerr << "lcqi test failure: " << error.what() << "\n";
407         return EXIT_FAILURE;
408     }
409 }

```

Listing 16.26 models/tiny_mlp_i8.txt

```

1 LCQI_MODEL_V1
2 input_size 4
3 hidden_size 3
4 output_size 2
5 hidden_scale 0.1
6 hidden_weights
7 5 -3 2 1

```

```
8 -4 6 1 -2
9 3 2 -5 4
10 hidden_bias
11 0.1 -0.2 0.0
12 output_scale 0.05
13 output_weights
14 4 -6 2
15 -3 5 8
16 output_bias
17 -0.5 0.4
```

Listing 16.27 models/tiny_tokenizer.txt

```
1 LCQI_TOKENIZER_V1
2 bos_id 0
3 eos_id 4
4 unk_id -1
5 chat_template_hash tiny-chat-template-v1
6 vocab_size 5
7 vocab
8 0 <bos>
9 1 tok1
10 2 tok2
11 3 tok3
12 4 <eos>
```

术语表

术语	实践含义
manifest	模型资产清单，记录 tensor name、dtype、shape、offset、checksum 和 tokenizer/配置版本。
shape verifier	在 kernel 运行前检查 shape、dtype、layout、量化 descriptor 和 state edge 是否匹配。
reference decoder	以清楚正确为目标的 decoder 实现，用来生成 trace 和 golden。
oracle	比较候选实现与 reference 的判定器，可使用精确相等或误差界。
trace checkpoint	推理链路中的可复查节点，至少包含 shape、stride、dtype、layout、checksum 和摘要值。
KV cache	attention 历史 K/V 状态，跨 token 存活，不能被普通 activation planner 复用。
packing	把权重改写成更适合热循环和 SIMD 的布局。
backend	执行计划落到机器的实现，例如 scalar、AVX2、AVX-512、GPU 或库调用。
fallback	后端能力不满足时显式降级到可验证路径，并记录原因。
SLO	服务目标，例如 TTFT、TPOT、queue wait、拒绝率和取消回收时间。